Revelio · position-coalescing 代码对照 + 流程图

明天 10/30 9:00–12:00 技术面用。仓库：~/Desktop/position-coalescing（VS Code 已打开）。每张图下面有「讲这张图时说什么」。图可以照着在 Google Doc 里重画。

讲解顺序（按这个顺序讲，约 35–40 分钟，留时间给他问）

1. **总体设计**：题目（图 0）→ 整条流水线（图 1）→ 每人一个状态机（图 2）
2. **数据告诉我什么 → 所以这么设计**（8 条数据发现）
3. **技术选型**：为什么 Polars、rapidfuzz、uv、Docker 这么配（1 分钟）
4. **匹配**：六条规则（图 3，点方框看详解）+ 打分算一遍（图 4）
5. **Scale**：64 桶、cgroup、Polars 线程、1000 亿行、Spark（图 7）+ 踩过的坑挑 2–3 个
6. **每天增量和迟到数据**（图 5）
7. **评估**：判官 + 人工 + 不用评委的检查 + 测试（图 6）
8. **试过的方案和取舍**
9. **To sum up**（含「为什么不让 LLM 判每一对」）→ **未来展望**
10. **怎么用 AI**（被问到再讲）

你自己做的决定不单独讲，散在每段里顺口带出来（每段里标了「你的决定」）。

图 0 · 题目：输入和输出

人话：同一个人的领英主页被抓了很多次，每次抓到的工作经历都不全、还可能改过字。要做的是：每个人、每个抓取日期，给出当天已知的最完整履历，每段工作一个永远不变的 id，而且不能用未来的信息。

```
输入（同一个人 user 7 的三次抓取）
asof        company      title               start
2024-01-05  Google       Software Engineer   2022-03
2024-03-10  Google       Senior SWE          2022-03      ← 改了头衔（Case B）
2024-06-02  Meta         ML Engineer         2024-05      ← 这次没抓到 Google（

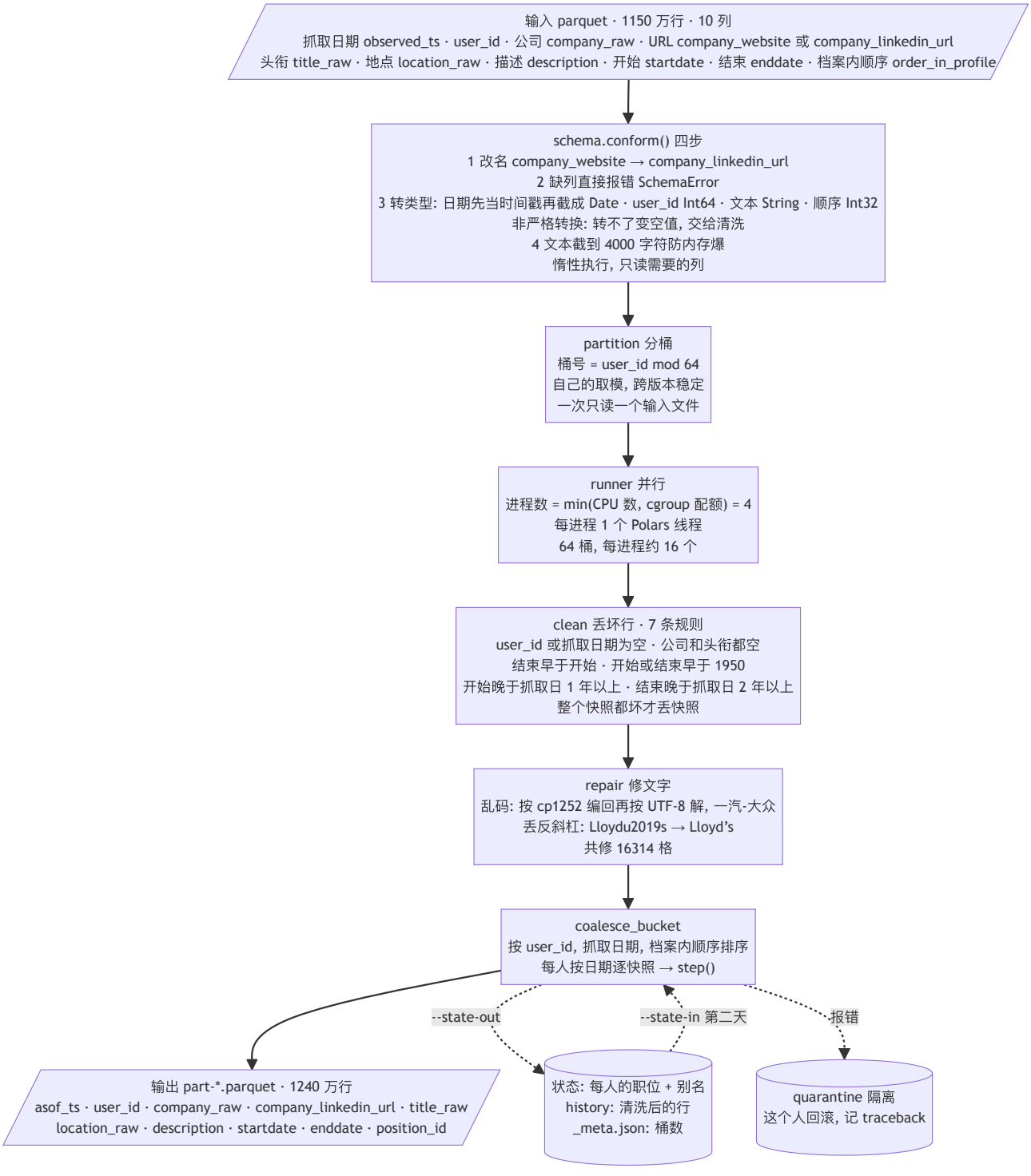
输出
```

```
asof        position_id   company   title              start
2024-01-05  7·2^16+0       Google    Software Engineer  2022-03
2024-03-10  7·2^16+0       Google    Senior SWE         2022-03   ← 同一个 id,
2024-06-02  7·2^16+0       Google    Senior SWE         2022-03   ← 没抓到也照样
2024-06-02  7·2^16+1       Meta      ML Engineer        2024-05
```

Case A: a scrape misses a position, so we carry known positions forward. Case B: the same position comes back edited, so we match it and keep the latest non-null value per field.

Case A = 漏抓了照样带着；Case B = 改了字要认出是同一段并用最新值。

图 1 · 整条流水线



讲这张图时说什么

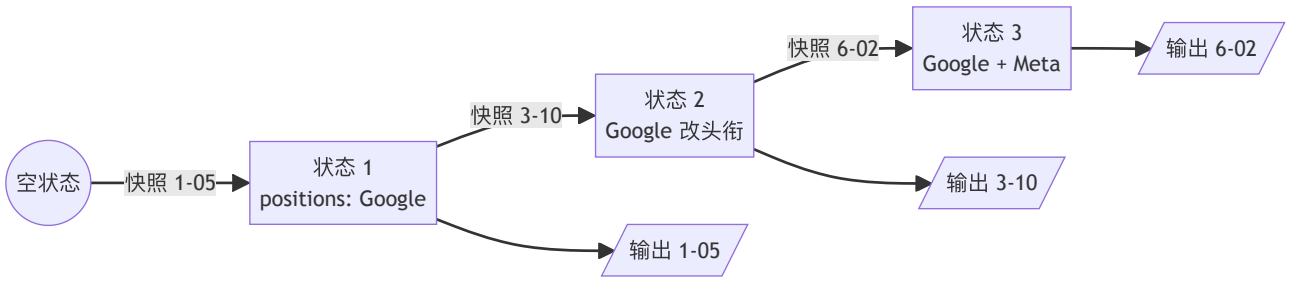
People are independent, so I hash users into 64 buckets and process buckets in parallel. Each bucket is cleaned, then every person is replayed snapshot by

snapshot in date order. The state at the end can be saved and continued the next day.

人和人互不相关 → 按 user_id 分 64 桶并行；桶里先清洗，再每人按日期逐个快照处理；最后的状态能存下来，第二天接着跑。

图上的框	代码	一句话
schema	`src/coalesce/schema.py` · `conform()`	统一列名（`company_website` 也接受），转类型，截断超长字段
partition	`partition.py` · `bucket_expr()`	`user_id % 64` 分桶
并行	`runner.py` · `run()`、`available_cpus()`	读 cgroup 的 CPU 配额，不用 `os.cpu_count()`（14 进程 5.5GB → 4 进程 3.7GB）
clean + repair	`clean.py`、`repair.py` · `repair_text()`	七条行规则丢坏行（README 写 six，是把「开始早于 1950」「结束早于 1950」算成一条）；乱码 `ä¸€æ±½` 还原成 `一汽`，修了 16,314 个格子
每桶	`bucket.py` · `coalesce_bucket()`	按人分组、按日期排序，逐快照调 `step`；一个人出错只隔离这个人
step	`step.py` · `step()`	核心：一个快照进 → 当天的完整履历出
入口	`cli.py` · `main()`	命令行参数：`--input --output --state-in --state-out`

图 2 · 每人一个状态机（为什么不会用到未来信息）



讲这张图时说什么

The step function only sees the current snapshot and the state built from earlier ones. So it can't use future information by construction, not by discipline. A test plants a future snapshot and checks it never shows up earlier.

step 只能看见「这一次的快照 + 之前攒下的状态」，结构上就不可能偷看未来；还有测试专门塞一个未来快照验证。

```
step.py
def step(state, observed_ts, rows):
    fp = fingerprint(rows)                    # 和上一次一模一样?
    if fp != state.last_fingerprint:          # 57.5% 的快照是重复的，直接跳过
        rows = dedupe(rows)                   # 规则 1
        assigned = match(rows, state.positions)   # 规则 2–5
        for i, row in enumerate(rows):
            pos = assigned[i] if i in assigned else state.new_position()
            pos.absorb(row)                   # 最新非空值覆盖
        state.merges += consolidate(...)      # 规则 6：折叠重复 id
    return [当天每一段已知职位]                # Case A：没抓到的也输出
```

状态里存什么（`state.py`）：每段职位的字段值 + 别名集合（用过的所有公司名、URL、头衔）；`next_seq`；上一次的指纹和日期。

id 怎么来：`user_id × 2^16 + 序号`。不用全局计数器，同样输入永远得到同样 id；id 只会退休（被折叠）不会改变。

数据告诉我什么 → 所以这么设计（README 第 2 节）

先看数据再定规则。每一条都是「数据里看到的现象 → 对应的设计决定」，这是整份作业最能体现思考的部分。

数据里看到的	所以这么设计
52% 的快照只有一行；30% 的人从来不超过一行	**Case A（沿用已知职位）是常态，不是特例**：一次抓取只抓到最上面一段，不代表别的工作没了，所以已知职位一直带着往后输出
57.5% 的快照和上一次一模一样	用**指纹**直接跳过，省掉一半最贵的匹配
7.9 万组「同雇主同开始但头衔不同」：一半是同一份写两遍，一半是真的并行职位	不能只看雇主 + 开始日期，**还要比头衔**
约 20% 的开始日期是 1 月 1 日	当成**「只写了年份」**（见下面）
有的抓取所有头衔都是空的（降级抓取）	**空头衔当通配，但只在这一行内有效**，不存成别名（存了就会吞掉以后所有头衔，Veeva 那个 bug）
URL 列在样例里叫 company_website	两个名字都收，输出一律叫 company_linkedin_url
乱码（ä¸€æ±½–å¤§ä¼— = 一汽–大众）和丢了反斜杠（Lloydu2019s = Lloyd's）	清洗时**无损修复**：cp1252 回转 + 转义解码，修了 16,314 个格子
坏数据不到 0.1% 的行	**只丢坏的那一行**；整次快照只有每行都坏才丢（390 万次里 2,539 次）

「只写了年份」具体怎么处理（match.py `start_compatible`）：很多人只填年份，系统存成那年的 1 月 1 日。所以比较两个开始日期时，如果**同一年、并且有一边是 1 月 1 日**，就当成兼容。例：一次写 2020-01-01（只填了年），一次写 2020-06-01（填了月份）→ 相差 5 个月，超过平时允许的 62 天，但因为是同一年且一边是 1 月 1 日，仍然算同一段的候选。只用在规则 2 找候选这一步。

About 20% of start dates are January 1, which usually means only the year was entered. So two starts in the same year count as compatible when either

one is January 1, even though they're more than two months apart.

你的决定（顺口带出来）：

- **只丢坏行，不丢整次快照**：你当时说「整次丢掉很可惜」。
- **输入文件是按行切的，不是按人切的**：同一个人的不同次抓取散在好几个文件里，所以一定要先按 user_id 分桶，才能把一个人的所有数据凑到一起。

Every rule came from the data: half the snapshots have one row, so carrying positions forward is the normal case; 57.5% repeat, so a fingerprint skips them; titles can be all null in degraded scrapes, so a missing title is a wildcard only for that row; and fewer than 0.1% of rows are bad, so I drop rows, not whole snapshots.

技术选型：为什么这么选（1 分钟）

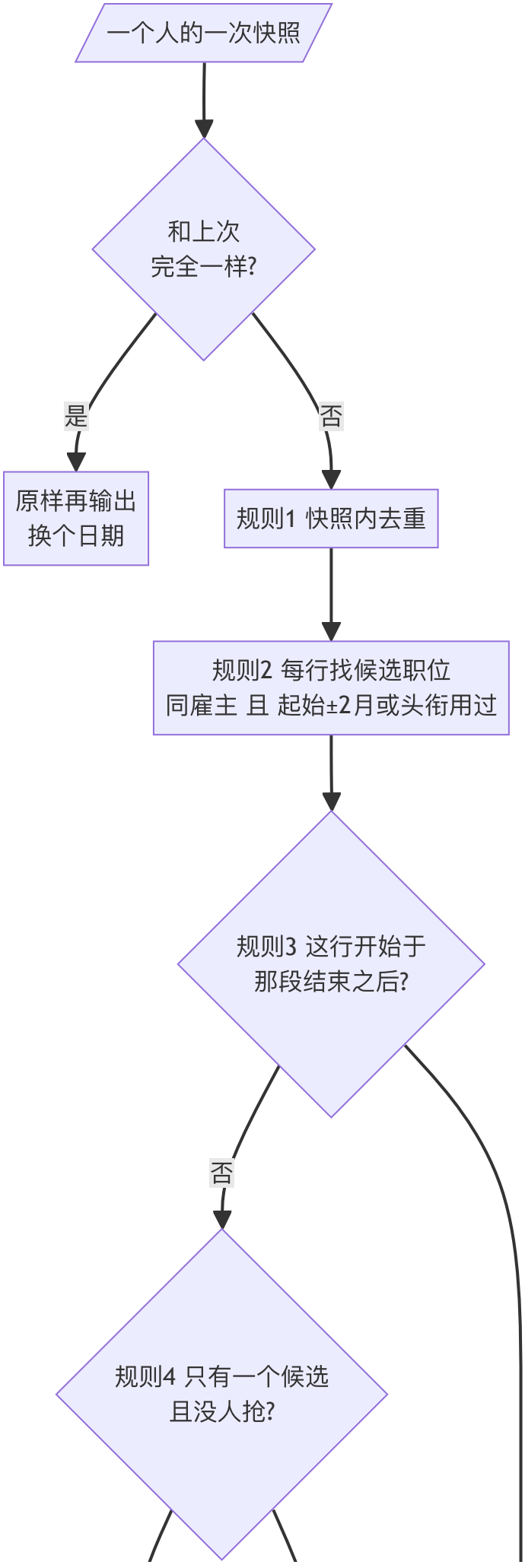
选项	好处	问题	结论
pandas	人人会	基本单核；字符串存成 Python 对象，2.5 GB 数据进内存会膨胀到十几 GB，4 核 16 GB 会爆	不选
DuckDB	SQL 好读、超内存自动落盘、很快	核心是「每人按时间一步步推进的状态机 + 模糊配对」，写成 SQL 很别扭，最后还得调 Python，两个引擎来回切	不选（纯 SQL 题会选它）
PySpark 本地模式	和生产同一套代码	JVM 启动慢、镜像 1 GB+，单机 1000 万行反而比 Polars 慢，而且评分要计时	交作业不选，生产选
Polars	Rust 多线程、惰性执行只读需要的列、Python 写法干净；和 Arrow 零拷贝，能直接交给 Spark	超内存落盘不如 DuckDB；版本间 API 变化快	**选**：落盘问题用「自己先按人分桶」解决，版本锁死

- **rapidfuzz**：C++ 写的字符串相似度库，token_set_ratio 就是它的，比纯 Python 快很多。
- **uv**：锁定所有依赖的精确版本（uv.lock），镜像每次构建都一样。
- **Docker**：基础镜像按摘要钉死、非 root 用户运行、输入只读挂载、可以 `--network none` 断网跑、不放任何密钥。
- **polars-lts-cpu**：普通版 Polars 需要 AVX2 指令，在 Mac 上模拟 x86 的 Docker 里会卡死；lts-cpu 版不依赖 AVX2。
- **混合写法**：能向量化的（读、清洗、分桶、排序）交给 Polars；只有逐人的状态机和模糊配对走 Python + rapidfuzz。

I chose Polars over pandas because pandas is mostly single-threaded and would blow up memory, and over DuckDB because the core is a per-person state machine with fuzzy matching, which is awkward in SQL. Spark is the production answer, but locally it's slower and the image is huge. Polars hands Arrow tables to Spark with zero copy, so the same function runs in both.

图 3 · 一个快照进来：六条规则

每一步的详解在这张图下面依次展开。


一个人的一次快照
和上次
完全一样?
是
否
原样再输出
换个日期
规则1 快照内去重
规则2 每行找候选职位
同雇主 且 起始±2月或头衔用过
规则3 这行开始于
那段结束之后?
否
规则4 只有一个候选
且没人抢?

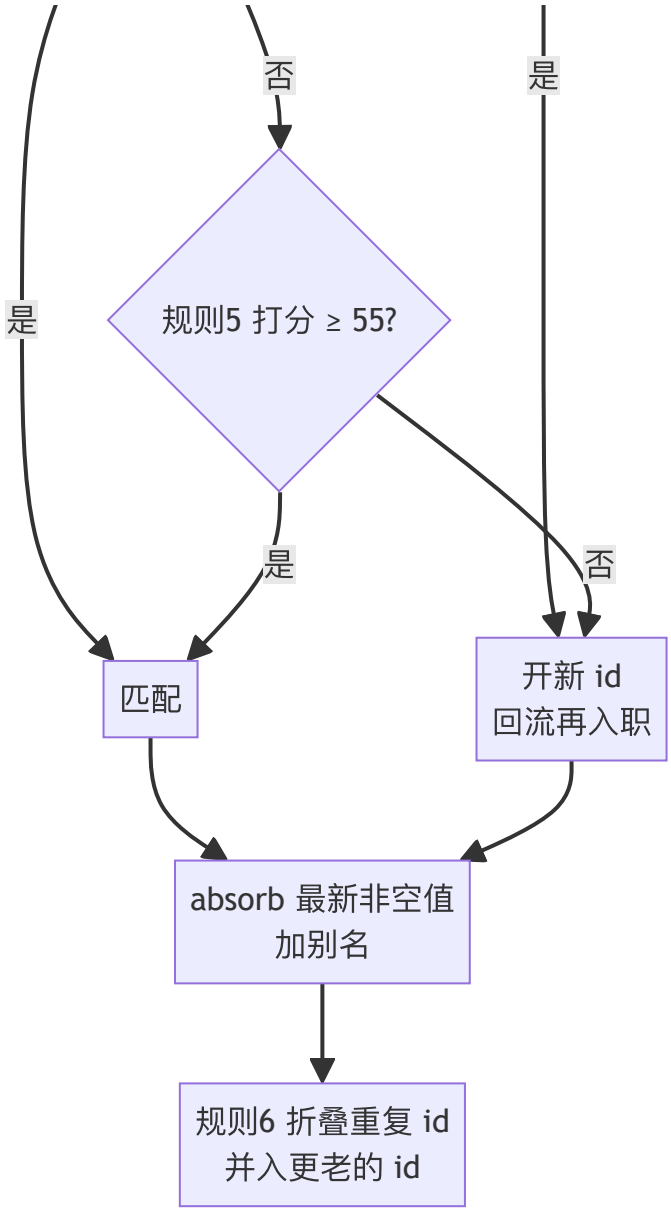
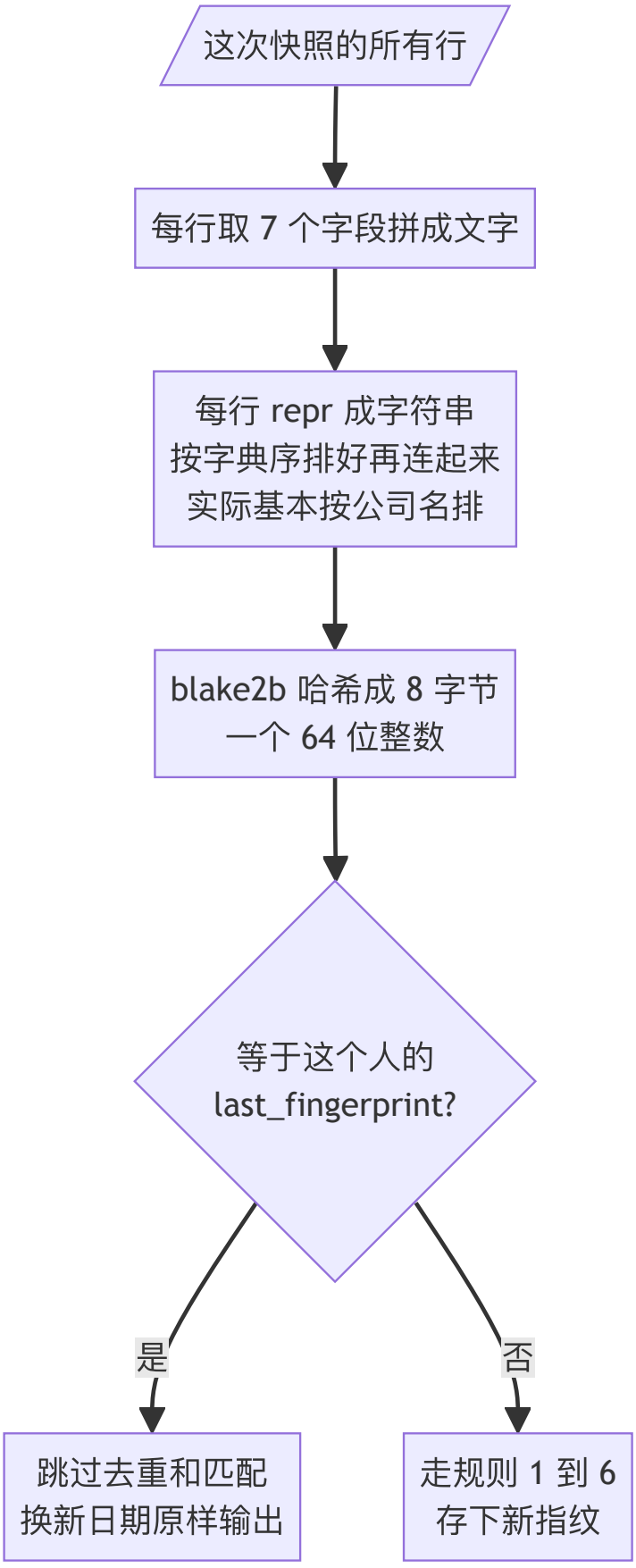


图 3 每一步的详解（打印版：原来点方框弹出的内容，全部展开放在这里）

和上次完全一样吗? · 指纹

step.py 第 16–40 行 · fingerprint()



- **为什么排序**：一次快照其实是「有哪几段经历」的集合，和顺序无关。但本人拖动调整顺序、或数据源每次返回顺序不同，都会让行的顺序变。不排序，内容没变也会算出不同指纹，白跑一遍匹配。
- **按什么排**：每行的 7 个字段（公司名、URL、头衔、地点、描述、开始、结束）先用 `repr()` 转成一串文字，例如 `('Google', 'linkedin.com/company/google', 'SWE', None, None, datetime.date(2022, 3, 1), ...)`，再按字符串字典序排。第一个字段是公司名，所以实际基本按公司名排，同名再比 URL、头衔……大写排在小写前。
- **顺序没有业务含义**：不按时间也不按重要性，只要求同样的内容不管原来什么顺序，排完都一样。用 repr 是因为一行里混着字符串、日期和空值，直接比 None 和字符串会报 TypeError。
- **跳过安全吗**：安全。内容一样说明值早已吸收进状态；顺序只影响平局谁先，重跑反而会让结果无故抖动。
- 57.5% 的快照和上一次一样，这一半直接省掉最贵的匹配。
- 不用 Python 的 `hash()`：每个进程启动时加随机盐，指纹要存进状态，第二天别的进程读回来比，必须每次算出同一个数。
- 撞号概率约 1/1.8×10^19；真撞了也只是这次被当成没变，下次有变化照常处理。

A 64-bit blake2b hash of the sorted rows. Same as last time means skip matching and re-emit with the new date, 57.5% of snapshots.

原样再输出，换个日期

- 状态不动，职位和 id 都不变，只把 asof_ts 换成这次的抓取日期，再输出一遍。
- 还会更新 last_ts（处理到哪天），这是判断迟到数据要用的。
- 为什么还要输出：题目要求每个抓取日期都有一份「截至当天」的完整履历。

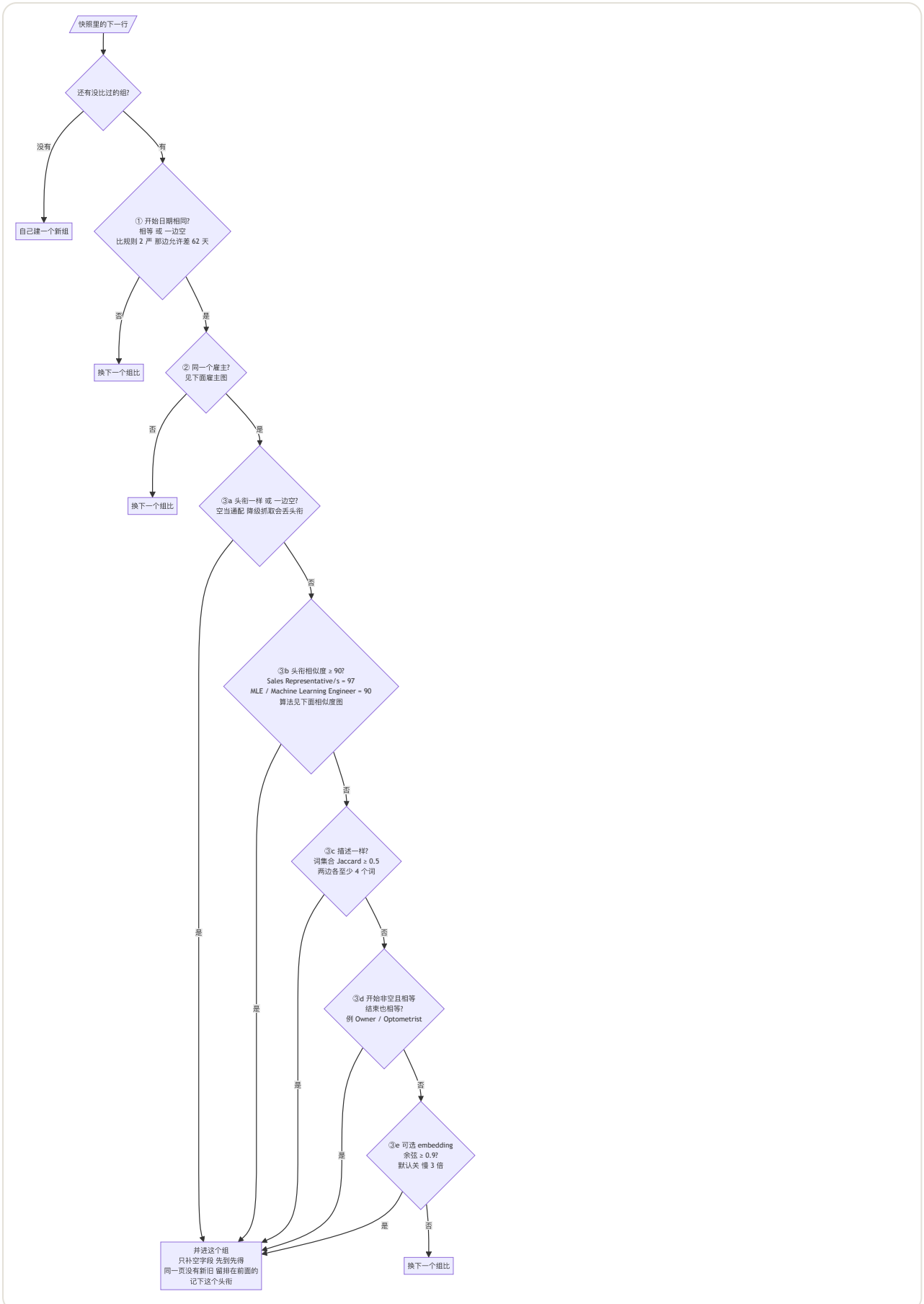
Nothing changed, so the same positions and ids are emitted again with the new as-of date.

规则 1 · 快照内去重

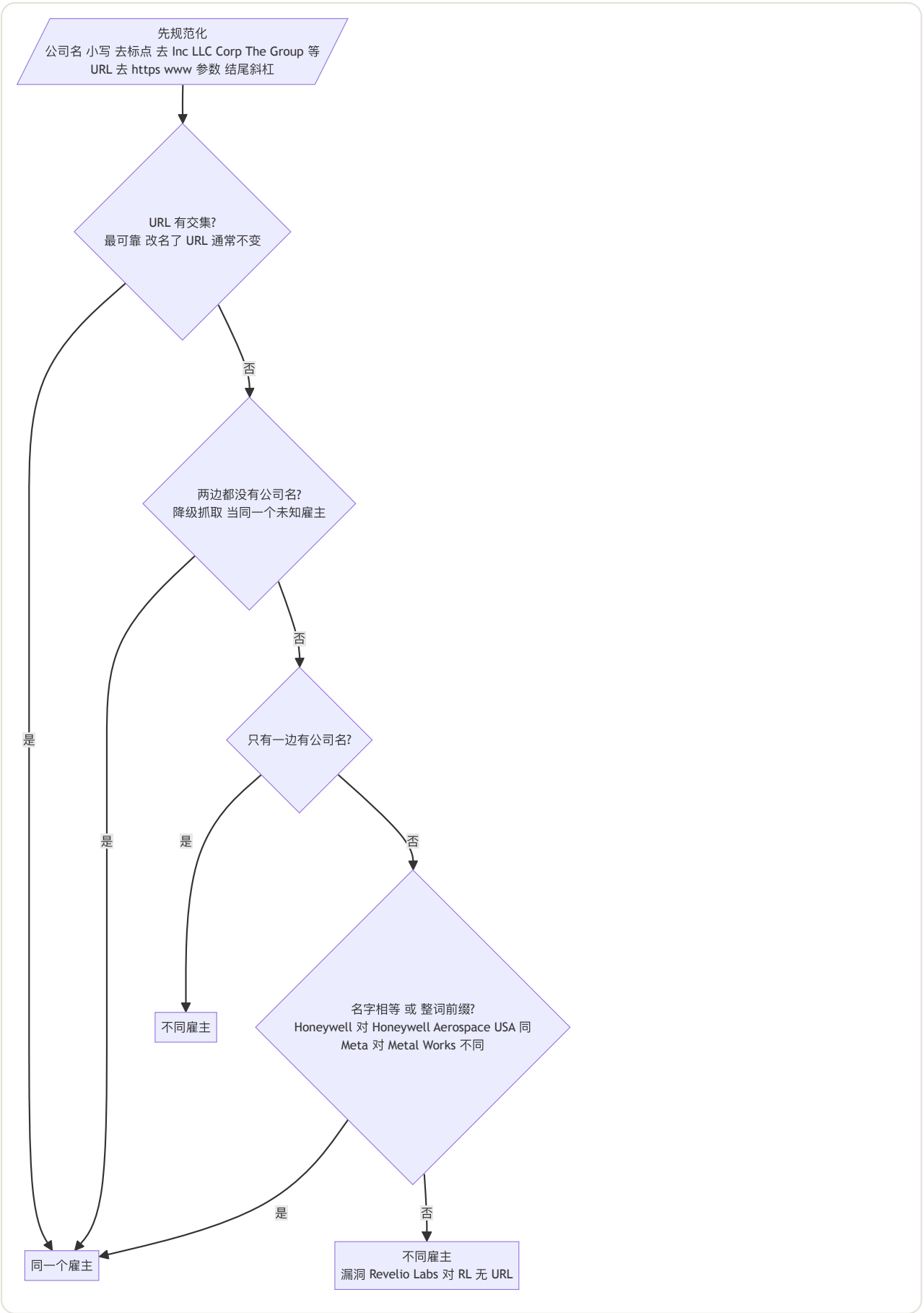
match.py 第 87–127 行 · dedupe() / _same_title()

为什么：同一次快照里真有人一个职位写两次：写法略不同（Sales Representative/s）、一份工作挂两个头衔（Owner / Optometrist）、两种语言各一遍、爬虫重复。不先并，后面会发两个 id。

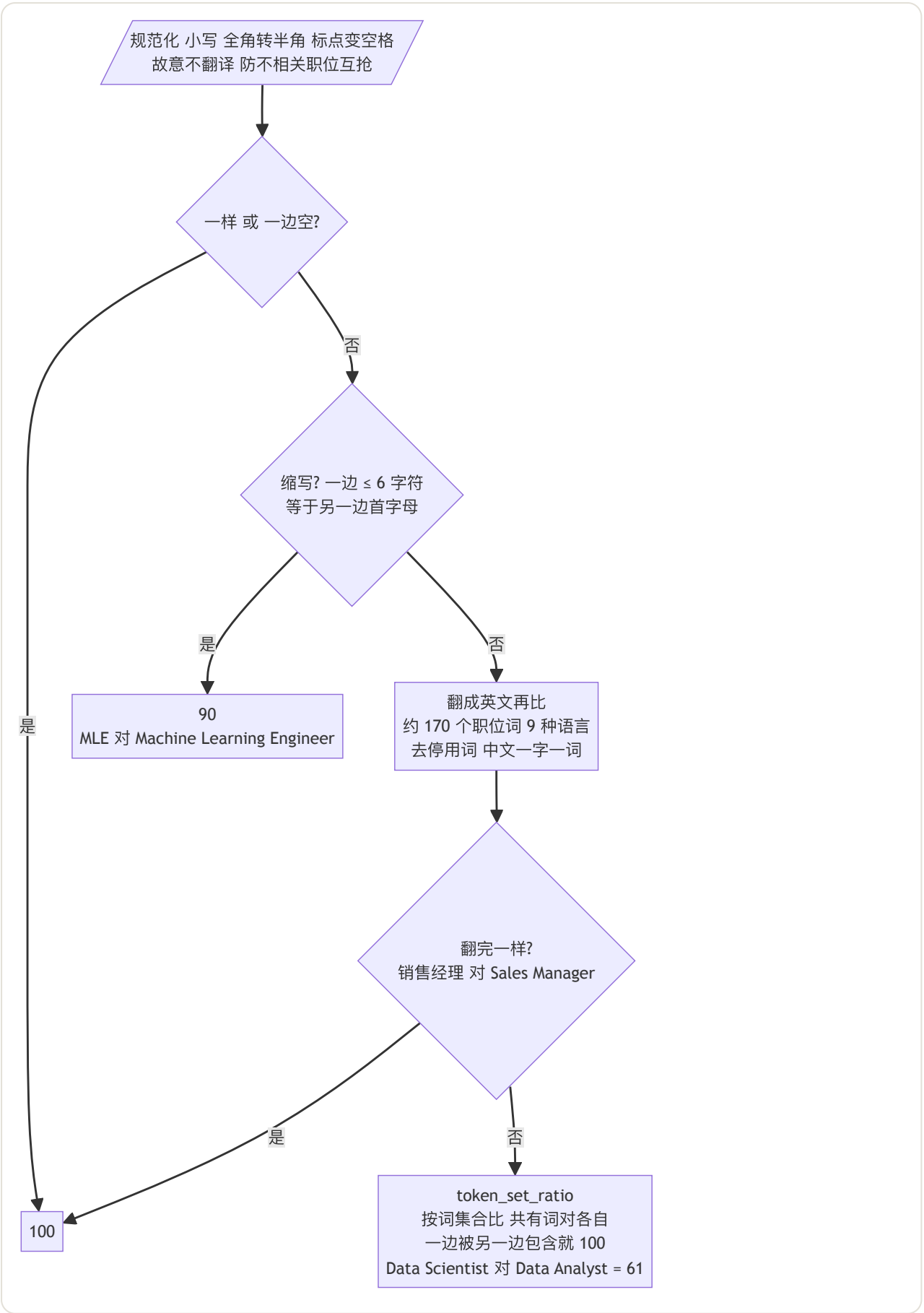
① **主流程**（每行跟已建好的组一个个比，①②③ 都过才并）



② 同一个雇主怎么判（employers_overlap）



③ 头衔相似度怎么算（title_similarity，0—100）



一句话方法：头衔先统一大小写和标点，把常见外语职位词翻成英文，再用 rapidfuzz 的 token_set_ratio 按词集合比，打 0–100 分；一边被另一边包含 = 100，缩写对上全称 = 90。

为什么头衔不用 Jaccard：头衔就两三个词，Jaccard（共有词 ÷ 全部词）太严，多一个词就掉一大截，还只认完全一样的词。

两个头衔	Jaccard	token_set_ratio
Software Engineer / Senior Software Engineer	2/3 = 67	100
Assistant / Admin Assistant	1/2 = 50	100
Sales Representative / Representatives	1/3 = 33	97

描述为什么反而用 Jaccard：描述是几十上百词的长文本，个别词差异影响小，词集合重叠刚好合适，字符级比较又慢。短的头衔用 token_set_ratio，长的描述用 Jaccard。

代价：包含就 100 有点宽（Engineer 对 Software Engineer 也是 100），所以它从不单独决定合并，必须同雇主、同开始。

Titles are two or three words, so Jaccard is too harsh: adding "Senior" drops it to 67 and a plural scores 33. token_set_ratio handles containment and small spelling differences. Long descriptions use word-set Jaccard, which works fine and is cheaper.

你的决定：「同雇主 + 同开始 + 同结束就合并」这条你一开始明确说「不对」，不让加（怕把真的并行职位合掉）。后来你亲手标了 60 对，发现这类大多是「升职了自己粉饰一下」或同一份写两遍，才加上。用数据改主意。

证据：人工标 15 对全同一份；多并 2,963 组，疑似漏配 10.5 万 → 5.5 万。**代价**：真同时做两份且日期一样会被并（Author / Psychologist），不并错约 2,200 组、并错约 680 组，选并。**故意不并**：开始同、结束不同、头衔不同 = 多半升职。

Merge rows in one snapshot with the same start, the same employer and a matching title before matching against known positions.

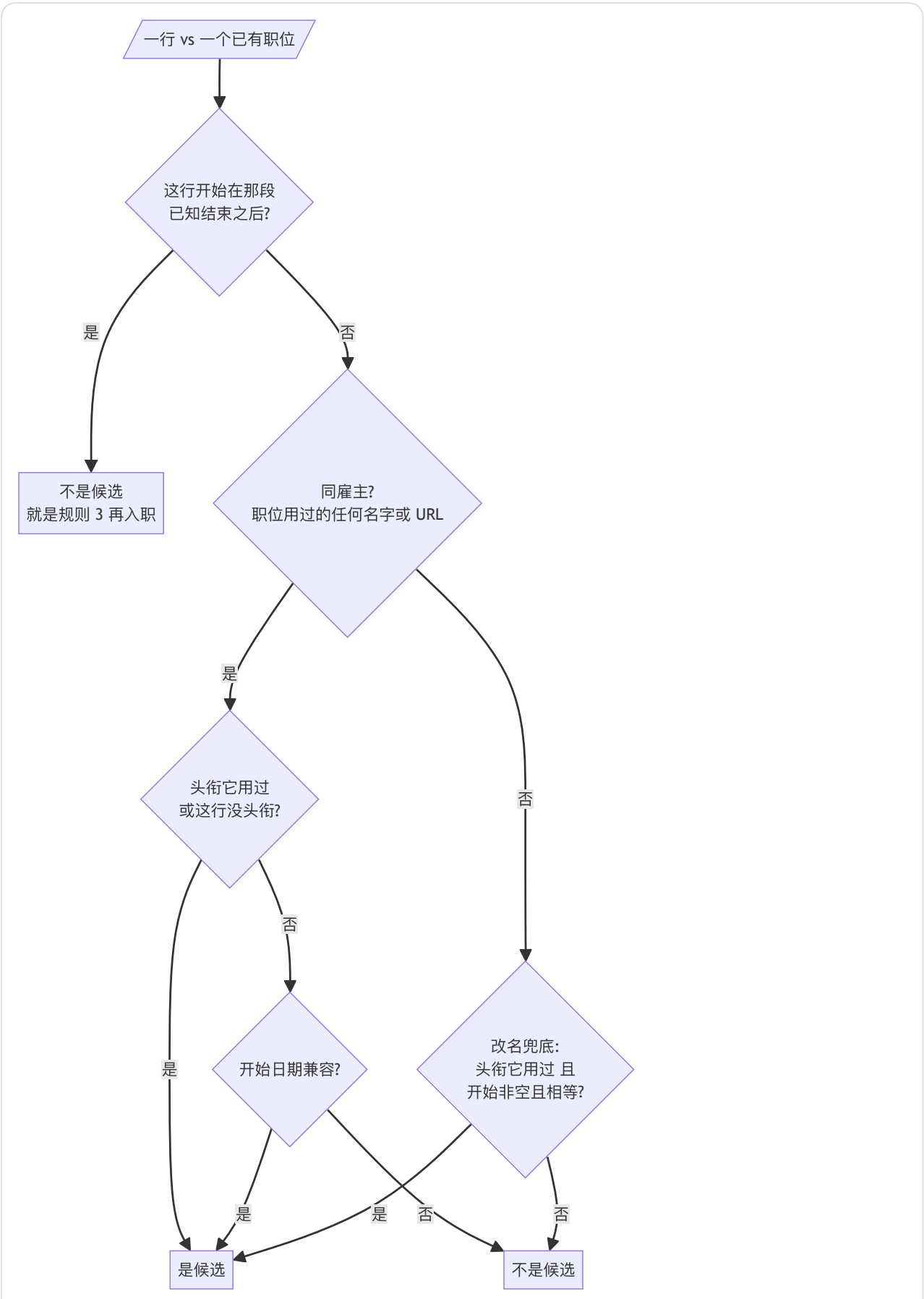
规则 2 · 每行找候选职位

match.py · is_candidate()　拿这一行和这个人每个已有职位逐个比

候选 = 「嫌疑名单」：不是「可以替代」，是**有可能是同一份工作、值得再比一比**的旧职位。像招聘先筛简历：规则 2 只做初筛，把明显不可能的排除，留下候选人；最后配哪一个，是规则 4、5 的「面试」。

已有职位	新行：Google · Senior SWE · 2022–03
Google · Software Engineer · 2022–03	同公司、开始一样 → **是候选**
Google · Intern · 2020–06	同公司，但开始差两年、头衔没用过 → 不是
Meta · MLE · 2024–05	不同公司 → 不是

只有 1 个候选又没人抢 → 规则 4 直接配上；好几个或被抢 → 规则 5 打分；一个都没有 → 开新 id。先筛候选是因为打分贵，也防止不同公司头衔像就被拿来比。



- **开始日期兼容**：相差 ⩽ 62 天（人会改一两个月），或有一边空，或同一年且一边只写了年份（存成 1 月 1 日）。
- **同雇主**看的是这个职位**历史上用过的所有**公司名和 URL（别名集合），不只看最新的。

- **改名兜底**：公司改名又没 URL，例：Polar Truck → Polar Transportes – Grupo DHL，头衔一样、开始同一天，就认。
- 只是「候选」，还没配上；配不配由规则 4、5 决定。

A position is a candidate if it's the same employer under any alias and either the title was used before or the start is within two months; or, for renames, the same title and the same start.

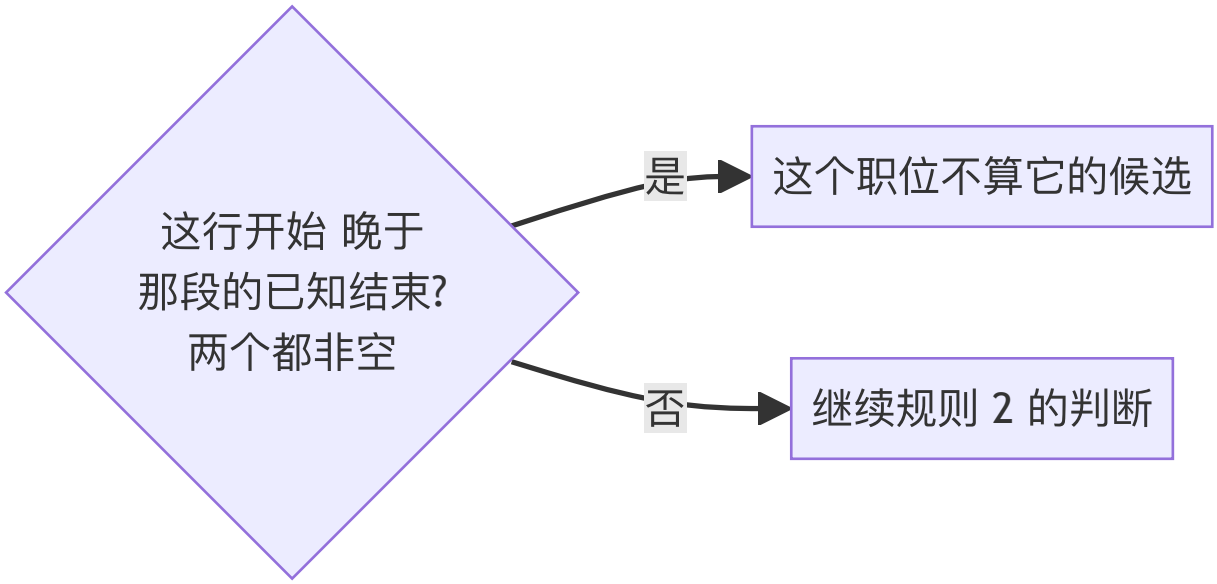
规则 3 · 再入职（回流）

match.py · starts_after_end()，在 is_candidate() 第一行就判断

不是 validation：数据完全正常，这条只是在决定「这一行属于哪个职位」。validation 是 clean 那一步（开始晚于结束、早于 1950 年这种坏数据，直接丢）。

你的决定：再入职开新 id 是你中途改的主意，原话「我现在感觉更对」。效果：多 1,579 个 id，939 个「日期乱跳」的 id 降到 0，抽查 57/58 对，还快了 5%。

大白话：人离职了又回同一家公司（boomerang）。**没有这条会怎样**：同公司、头衔也用过 → 被当成候选 → 规则 4 一看没人抢直接配上 → 两段并成一个 id，absorb 把开始改成 2022、结束改成空，2019–2020 那段被抹掉。

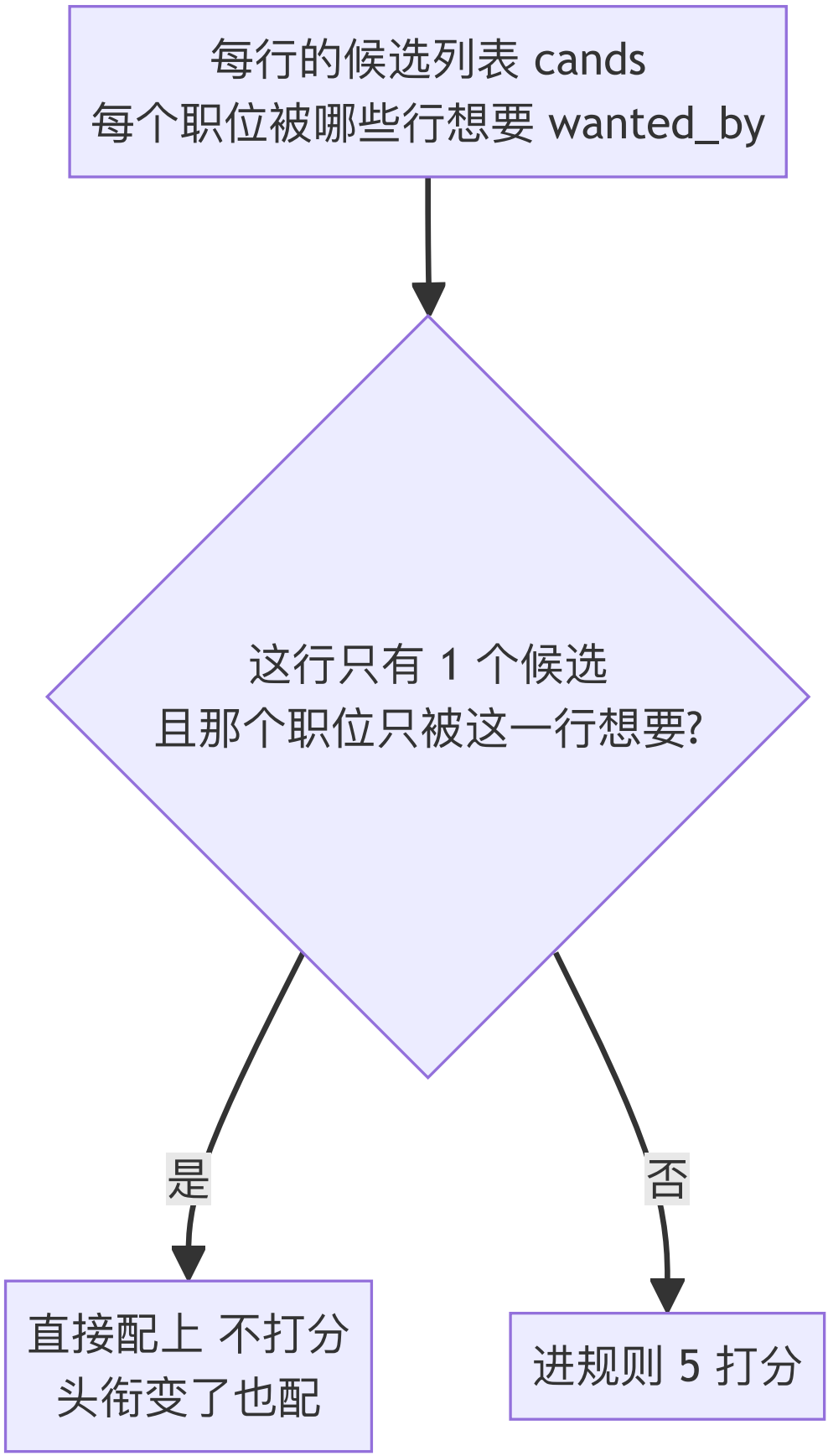


- 例：2019–2020 在 Starbucks 当 barista，2022 又回去当 barista。头衔、公司都一样，但时间上不可能是同一段，所以开新 id。
- 常见于季节工、回流员工。
- 只在两个日期都有值时才判断；结束日期空的（在职或不知道）不拦。
- 注意：它只是把这一个旧职位排除掉，这行仍可能配上别的职位；都配不上才开新 id。

A row that starts after a position's known end is a rehire, never an edit of that position.

规则 4 · 只有一个候选，且没人抢

match.py · match() 第一轮（代码注释里叫 Rule 3）

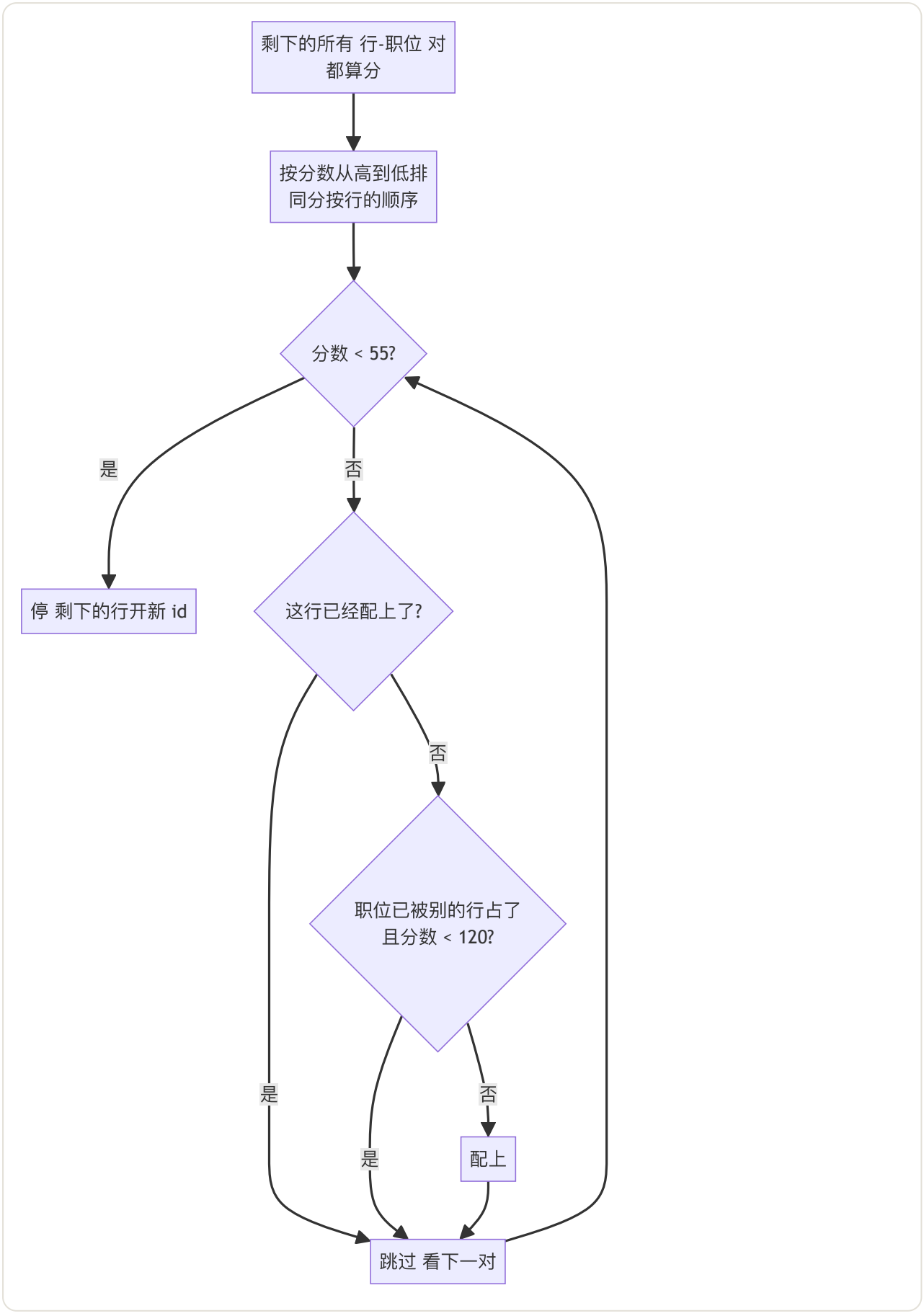


- 这是 Case B（改头衔）的主力：Machine Learning Engineer → MLE，Software Engineer → Senior SWE，在同公司又没人抢，就是同一段被改了。
- 体现 merge when in doubt：没有别的解释时，宁可认成同一段。
- 一对一：一行只配一个职位，一个职位这一轮只给一行。

If a row has exactly one candidate and no other row wants it, match outright without scoring. That's how edited titles keep their id.

规则 5 · 有争抢时打分（≥ 55 才配）

match.py · _score() + match() 贪心



- **头衔相似度怎么算**：和规则 1 同一个函数 title_similarity：规范化 → 外语词翻成英文 → rapidfuzz token_set_ratio（不用 Jaccard 的原因见规则 1 弹窗）。
- **分数** = 头衔相似度（和职位用过的头衔取最高，0–100）+ 同雇主 25（头衔和别名完全一样再 +2）+ 开始相同 10 + 结束相同 5。
- **55**：同雇主时头衔相似度至少要 30；不同雇主几乎配不上。
- **120**：头衔一模一样 + 同雇主，说明是爬虫重复抓，即使职位已被占也并进去。
- 什么时候会争：同公司同时有两个职位（并行岗），或一行同时像两个旧职位。
- 例（实测）：已知 Google · Software Engineer · 2022–03，来了 Senior Software Engineer · 2022–03 → 100 + 25 + 10 = **135**，配上。

Contested pairs are scored as title similarity plus 25 for the same employer, 10 for the same start and 5 for the same end, then paired greedily from the top. Below 55 opens a new position; 120 or more can join an already taken one.

匹配

- 这一行沿用那个职位的 id，然后进 absorb 更新字段。
- id 一旦发出去就不再改（除了规则 6 合并时，新的并进老的）。

开新 id

state.py · new_position()

- id = user_id × 2^16 + 这个人的下一个序号（next_seq），然后 next_seq + 1。
- 不需要全局计数器，结果可复现；不同人高位不同，不会撞号。
- **你的决定**：借鉴系统设计那本书第 7 章 Snowflake 的思路：把 64 位拆成几段、各段各管各的，不需要中心协调就能保证唯一。这里只拆两段：高位 user_id，低 16 位每人自己的序号。
- 一个人超过 65536 个职位就报错进隔离，绝不悄悄撞号（数据里最多 166 个）。

New ids are user_id shifted left 16 bits plus a per-user sequence, so no global counter is needed.

absorb · 更新字段 + 加别名

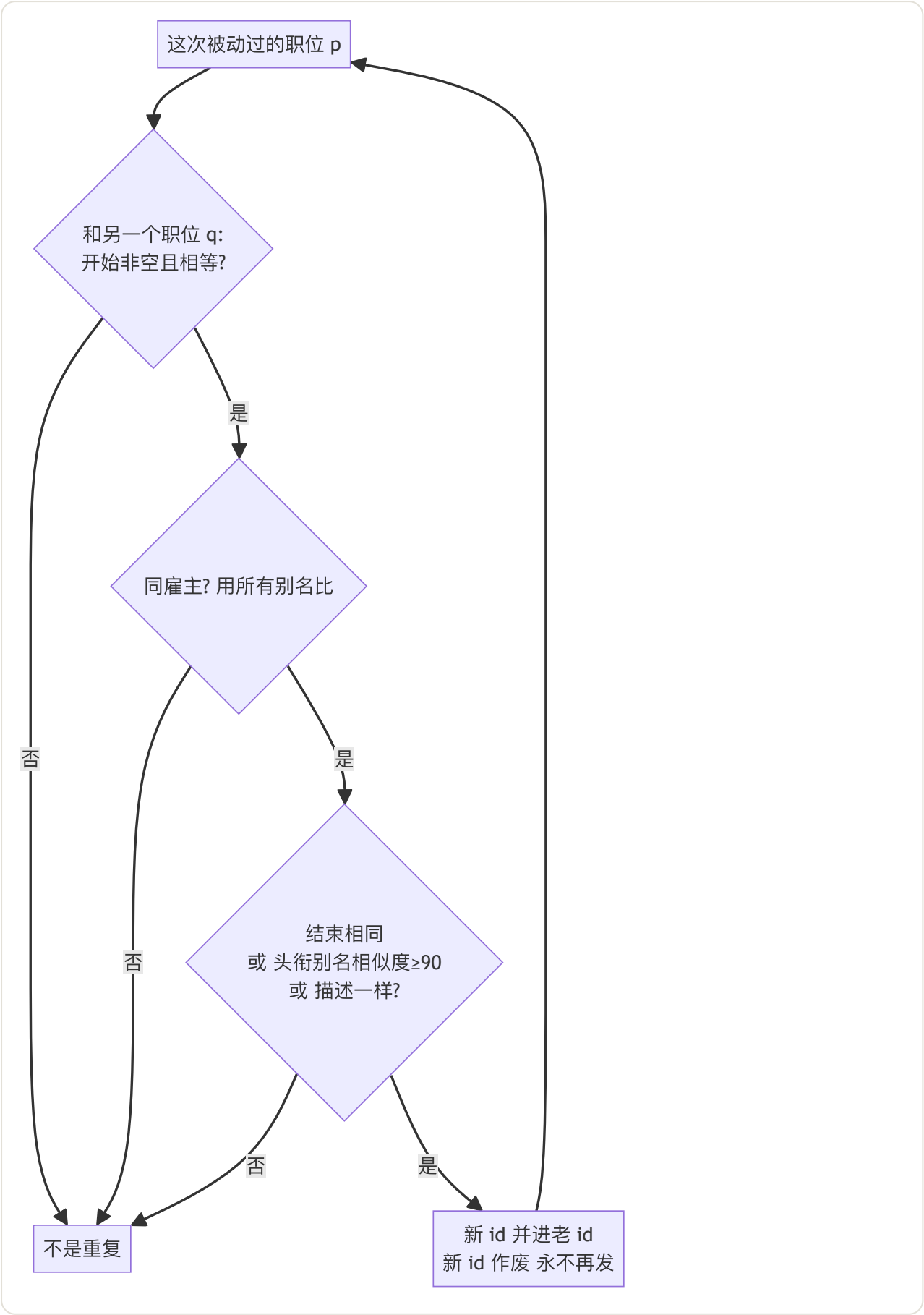
state.py 第 43–68 行

- 每个字段：这次有值就用这次的（最新非空值），这次空的保留旧值。
- **结束日期特别处理（你的决定，先想错再用数据纠正）**：一开始你主张「空的结束日期 = 至今/在职」，所以新来一行结束是空，就把旧的结束日期也清空。跑完一看：43,356 个已知的结束日期被抹掉了（Veeva 那个人 2019-12 的结束被一次降级抓取抹成空）。原因是抓取经常丢字段，空更可能是**「没抓到」**而不是「还在职」。于是定成：**空 = 没信息**，不覆盖已知值；结束日期只取合法的非空值（先用这次的，没有就留上次的）。修完抹掉的降到 53 个，剩下的是再入职，后来由规则 3 解决。**这个认知的好处**：和题目「每个字段取最新非空值」完全一致；避免一次坏抓取把历史擦掉；也不会拼出开始晚于结束的日期。
- **英文**：I first treated an empty end date as "still ongoing" and it wiped 43,356 known end dates, because a degraded scrape drops fields. An empty end means "no information", so it never overwrites a known end. Erasures dropped to 53.
- 别名累加：公司名、URL、头衔都加进集合，以后改名、改头衔还能认出来。
- 空头衔不当别名：否则一次降级抓取会让这个职位以后吞掉同公司所有行（数据里真出现过：2018 年的 Consultant 吞了 2024 年的 Sr. Director）。

Latest non-null value wins for each field, aliases accumulate, a missing end date never erases a known one, and a missing title is never stored as an alias.

规则 6 · 折叠重复 id，并进更老的

match.py · consolidate() / same_position()（代码注释里叫 Rule 7）



- **为什么前面的规则管不了**：规则 1 只管同一次快照内的行；规则 2–5 是「一行配一个旧职位」，一对一，没办法把两个已经发出去的 id 并成一个。一个 id 发错了，以后每次这行只会配上其中一个，另一个一直挂着，永远配不回去。
- **为什么会发错**：每个职位的别名是慢慢攒起来的。早期快照里信息少，两个 id 当时看不出是同一份。后来有快照补上了 URL、头衔这些信息，把两边连了起来，这时候才看得出来。规则 6 就是在这个时候「回头补救」。
- **大白话**：前面的规则是「新来的人分到哪个房间」；规则 6 是「发现两个房间住的其实是同一个人，把房间合并」。
- **效果**：全量 965 次合并，疑似重复组 4.9 万 → 3.7 万，判官抽查 52/64 对（约 81%）；错的是同起止的两个角色（两门课的助教），和规则 1 同一种取舍。
- **代价**：承诺从「id 永远不变」改成「id 可因合并退役，退役后永不复用」；只查这次被动过的职位，开销小。
- 就是把规则 1 的判断用在两个已有职位上；只查这次被动过的，不用每次全比。
- 字段取最近更新的那一个，别名取并集。
- 例：Jeppesen 那个人一段工作被拆成 5 个 id，折叠后变 1 个。

After applying a snapshot, two positions that rule 1 would have merged are folded into the older id, and the newer id is never reissued.

规则	代码（match.py）	人话
1 去重	`dedupe()`	同一次抓取里同雇主、同起始，且头衔一样/相似 ⩾90/描述一样/结束也一样 → 是同一条
2 候选	`is_candidate()`	同雇主（名字规范化后相等或前缀、或 URL 相同）且（起始 ±2 个月或只有年份，或这个头衔它用过）。改名兜底：头衔完全一样 + 起始同月
3 再入职	`starts_after_end()`	这行开始日期在那段已知结束之后 → 不可能是同一段
4 无争议	`match()` 第一轮	只有一个候选、而且这个候选也只被这一行想要 → 直接配上（哪怕头衔变了，就是 Case B）
5 有争议	`_score()` + `match()` 贪心	按分数从高到低配对；低于 55 开新 id；⩾120 可以进已被占的职位（抓取重复）

6 折叠	`consolidate()`、`same_position()`	以前发出去的两个 id 其实是同一段 → 并进更老的 id（Jeppesen 那人一段工作被拆成 5 个 id）

Matching is greedy by score: title similarity from 0 to 100, plus 25 for the same employer, 10 for the same start and 5 for the same end. Below 55 the row opens a new position.

打分 = 头衔相似度 + 同雇主 25 + 同起始 10 + 同结束 5；低于 55 开新职位。

图 4 · 打分算一遍（被问「为什么 55」时用）

```
已知职位 P: Google · "Software Engineer" · 2022-03 · 结束 无

行 r1: Google · "Senior Software Engineer" · 2022-03
  头衔相似度（token-set）    ≈ 90
  同雇主                      +25
  同起始                      +10
  总分                         125  → ≥120，非常确定是同一段

行 r2: Google · "Product Manager" · 2023-07
  头衔相似度                  ≈ 20
  同雇主                       +25
  起始不同                    +0
  总分                          45  → <55，开新 id（内部转岗算新的一段）

行 r3: Alphabet · "Software Engineer" · 2022-03（没 URL）
  不同雇主 → 只走改名兜底：头衔完全一样 + 起始同月 → 候选
  头衔 100 + 同起始 10 = 110 → 配上（公司改名）
```

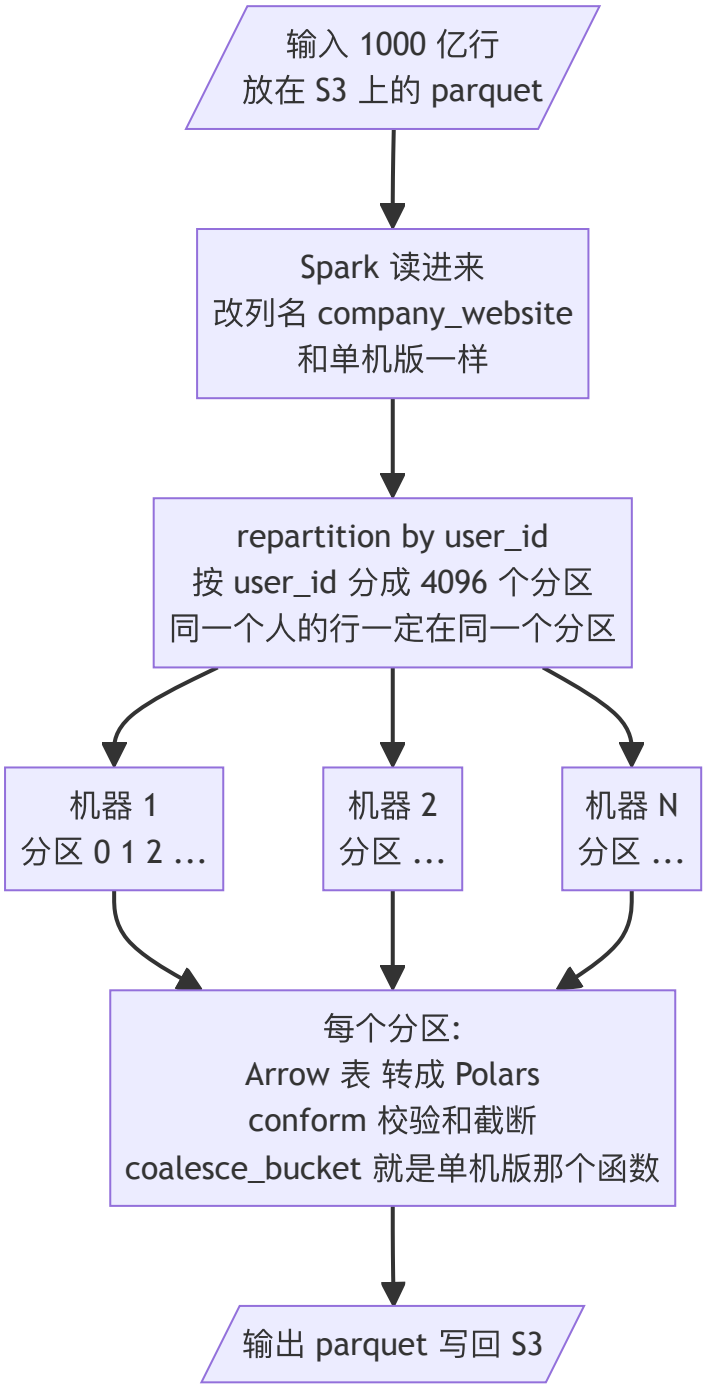
I chose to merge when in doubt. A missed merge creates a duplicate that is carried forward forever, while a wrong merge loses one title, and in the judged samples the first kind of error was more common.

拿不准就合：漏合会生一个永远带着的重复 id；合错只丢一个头衔；评委样本里前者更常见。

头衔相似度怎么算（`normalize.py` · `title_similarity()`、`glossary.py`）：token-set ratio；比之前先查一张小词表（glossary.py，约 170 个常见职位词），把外语职位词换成英文，比如西班牙语 gerente → manager、中文 经理 → manager。中文、日文词之间没有空格，就一个字算一个词，这样「销售经理」里也能找到「经理」，就像「project manager」里有「manager」。（韩文词表里有，但代码的判断范围漏了韩文字符，实际没生效，见讲解 flow 的局限。）`Gerente de Ventas` 和 `Sales Manager` 得 100。

图 7 · 扩到 1000 亿行

大白话：单机版是「一个人在家分 64 个桶处理」，Spark 版是「同样的活分给几百台机器，每台拿几个桶」。**分桶的方式和每个桶里干的活完全不变**，变的只是外面谁来分发、谁来干。



逐个名词

- **线性扩展**：人和人之间没有任何共享的东西，每个人单独算。所以数据翻一倍，总工作量就翻一倍；机器翻一倍，时间就减半。不会因为数据变大，出现「每个人都要和其他所有人比」这种越来越慢的情况。
- **约 800 核时**：实测单机 4 个进程，1150 万行约 78 秒 → 每个 CPU 核每秒约处理 3.5 万行。1000 亿 ÷ 3.5 万 ≈ 286 万核秒 ≈ **800 核时**（一个核要干 800 小时）。分给 64 个核同时干 ≈ **12.5 小时**；给 640 个核 ≈ 1.25 小时。
- **Spark 适配器 70 行**：`spark/coalesce_spark.py` 只有约 70 行，只负责「外层」：读数据、按 user_id 分区、把每个分区交给 `coalesce_bucket`、把结果写回去。匹配规则、id、状态一行都没写在这里。
- **跑同一个函数**：单机版和 Spark 版调用的是**同一个** `coalesce_bucket`，规则只有一份，改一处两边都生效，不会出现两套逻辑结果不一样。
- **mapInArrow**：Spark 的一个接口，把每个分区的数据以 Arrow 表的形式交给一个 Python 函数，函数返回 Arrow 表。Arrow 是列式内存格式，Spark 和 Polars 都认，所以在两者之间传数据几乎不用拷贝转换。
- **在 2 个文件上逐行一致**：拿 2 个输入文件，单机版跑一遍、Spark（本机模式）跑一遍，把两份输出逐行比，**每一行都一模一样**（73 万多行）。证明 Spark 外壳没有改变结果。

要主动说的局限（面试官问到就照实说）：

- 没在真集群上跑过，只在本机 Spark 上验证过。800 核时是按单机速度推算的。
- Spark 版目前只做一次性全量：增量状态和隔离没接进去，代码注释里写了接法（当天数据按 user_id 和状态表 join，每个分区调 `coalesce_bucket(df, states)`，再 MERGE 回状态表）。

  The Spark version only does a one-off full run so far; incremental state and quarantine aren't wired in yet. The plan is to join each day's data with the state table on user_id, call coalesce_bucket(df, states) per partition, and MERGE the new state back.
- 每个分区是整个读进内存的。1000 亿行 ÷ 4096 个分区 ≈ 每个分区 2,400 万行，会太大；真跑要把分区数调到每个分区几十万行（几十万个分区），和单机版每桶约 18 万行同一个量级。
- 生产里状态应该放**按 user_id 分区、只追加**的表（比如 Iceberg / Delta），而不是现在的 parquet 文件。**为什么**：

1. **现在每天整份重抄**：--state-in 读昨天的文件夹，--state-out 写一个全新的文件夹，64 个桶每个都整份重写，哪怕今天只有 1% 的人有新数据。用表就可以只更新（MERGE）今天动过的那些人。比方：现在是每天把整本账抄一遍，表是只改动过的那几页。
2. **按 user_id 分区 = 只读需要的**：今天只有某些人有新数据，按 user_id 分区后只读这些人所在的分区（分区裁剪），不用把全部状态读进来；和当天数据 join 时也天然对齐。
3. **整次提交要么全成功要么全不算**：现在是每个文件单独原子写，64 个文件之间没有整体保证，跑到一半挂了会新旧混在一起（现在靠写到新文件夹规避）。Iceberg / Delta 有事务：一次运行整体提交，读的人永远看到完整的一版。
4. **旧版本都留着（时间旅行）**：只追加 = 不覆盖旧数据，每次提交是一个新版本。某天跑坏了可以直接回退到昨天的状态；也能查「某一天我们知道的是什么」，正好对上 as-of 的需求和审计。
5. **其他**：加字段不用重写全表（schema 演进）；小文件自动合并；多个任务同时读写不会读到一半的数据。

Today each run rewrites every bucket's state into a new folder, even if only 1% of people changed. A table partitioned by user_id lets a run MERGE only the touched users and read only their partitions. Iceberg or Delta also gives one atomic commit per run instead of 64 separate files, and keeps old versions, so a bad run can be rolled back and we can ask what we knew on any date, which fits the as-of model.

People are independent, so the work grows linearly. Measured at about 35 thousand rows per core-second, 100 billion rows is roughly 800 core-hours, about 12 hours on 64 cores. The Spark adapter is about 70 lines: it repartitions by user_id and calls the same coalesce_bucket through mapInArrow, so there's one copy of the logic. On two input files it matched the single-machine output row for row. I haven't run it on a real cluster, incremental state isn't wired into it yet, and at that scale I'd raise the partition count so each partition stays around a few hundred thousand rows.

Scale 段：工程上踩过的坑（挑 2–3 个讲，推荐前三个）

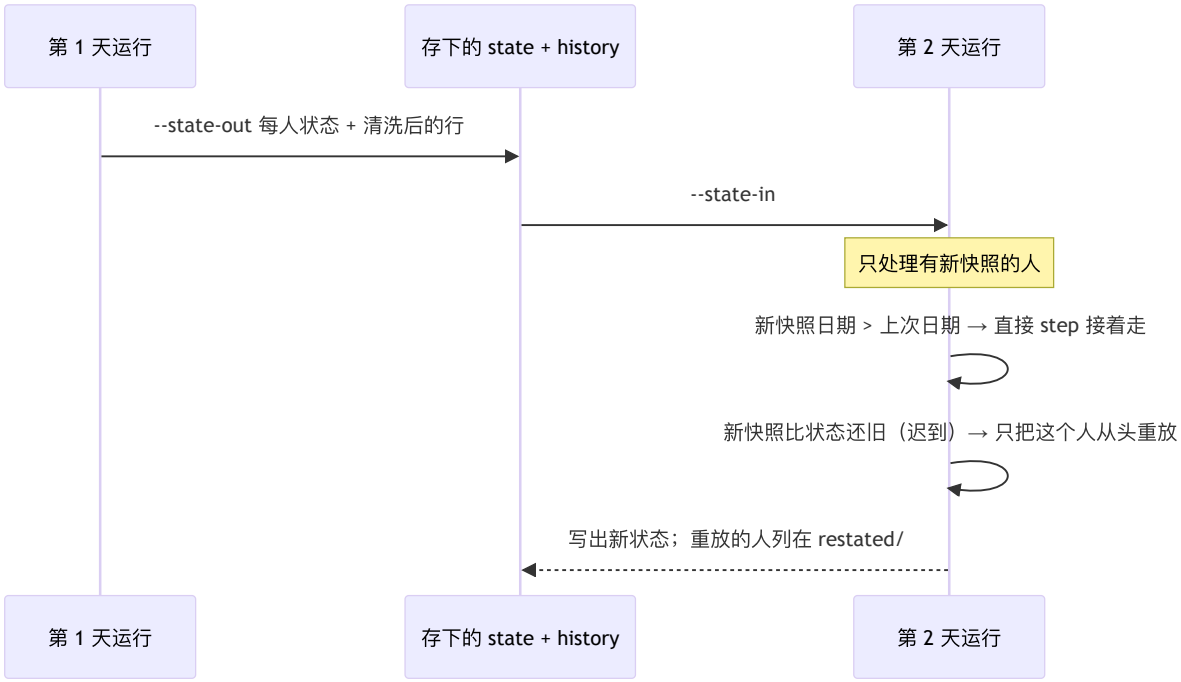
坑	怎么修	前后

线程抢占：4 个进程每个又开满线程，CPU 大部分时间在切换（系统时间 885 秒）	每个进程只给 Polars 1 个线程（POLARS_MAX_THREADS=1，在开进程池之前设）	104 秒 → 65 秒
容器限了 4 核却看到 14 核：`os.cpu_count()` 读的是宿主机	读 cgroup 的 CPU 配额（/sys/fs/cgroup/cpu.max）	14 进程 5.5 GB → 4 进程 3.7 GB
32 桶在 7.6 GB 的 Docker 里爆内存，只输出了 75 万行	改 64 桶，每桶减半；处理完立刻释放	之后没挂过，峰值约 2 GB
普通 Polars 在模拟 x86 下卡死	换 polars-lts-cpu	Docker 里约 190 秒，输出和本机完全一致
合并后出现开始晚于结束	absorb 里加日期保护：结束只取合法的非空值	733 个 → 0
Spark 执行器用了系统自带的旧版 Polars	指定 venv 的 Python（PYSPARK_PYTHON）	和单机逐行一致

要求表里的：题目给 4 核 16 GB 跑 Docker。实测本机 75–78 秒、约 2 GB；Docker 在 Mac 上模拟 x86 约 190 秒，输出和本机完全一致。

Three fixes mattered: one Polars thread per worker took it from 104 to 65 seconds; reading the cgroup CPU quota instead of os.cpu_count cut 14 workers and 5.5 GB down to 4 and 3.7; and 64 buckets instead of 32 stopped an out-of-memory crash in a 7.6 GB container.

图 5 · 每天增量和迟到数据



Daily updates only touch people with new snapshots. If a snapshot arrives late, older than the saved state, I replay just that person from their history, so the result equals a full run. A test splits the real data into two days with 128 late people and checks the output matches a single full run.

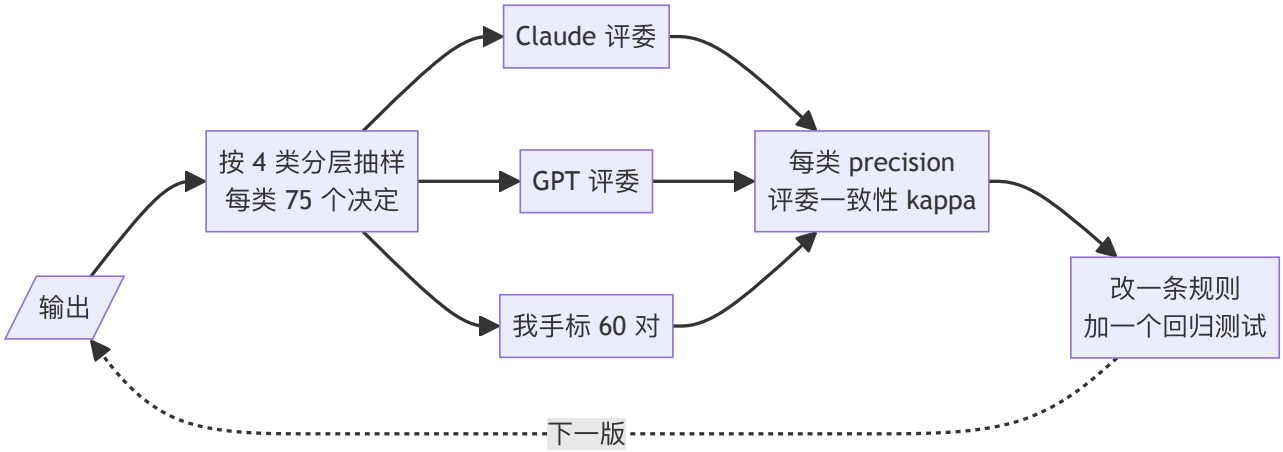
只动有新数据的人；迟到的数据只把那一个人从头重放，结果和全量跑一致，有测试验证。

你的决定：迟到数据要重放，不是隔离。一开始迟到的快照直接进隔离区，等人工处理。为什么改成重放：① 迟到很常见（爬虫重试、每周打包的数据源），隔离会积一大堆要人看；② 隔离意味着这个人在那几天的输出一直是错的，没人补；③ 重放成本低，只重跑这一个人，别人不受影响；④ 正确性可以验证：128 个迟到的人重放后和全量跑一模一样。代价是要多存 history，和这个人的所有行要重发一遍（restated 名单告诉下游替换）。

Late snapshots were quarantined at first. I changed that to replay because late data is routine, quarantine leaves that person's output wrong until someone looks, and replaying one person is cheap and verifiable: 128 late users replayed matched a full run. The cost is keeping history and re-emitting that person's rows.

代码：`bucket.py` · `coalesce_bucket()` 里的 `late` 判断和 `_replay()`；一个人报错就回滚到他进来时的状态（`checkpoint`），记进 quarantine。

图 6 · 没有标准答案怎么评估



4 类决定	Claude	GPT	人工
合并了，头衔大改	95%	95%	15/15
合并了，头衔中改	97%	97%	15/15
同雇主开了新 id	97%	87%	11/14
同雇主同起始却分开（弱点）	38%	8%	0/15

There's no ground truth, so I sampled 75 decisions from each of four strata and asked two LLM judges, plus 60 pairs I labeled myself. The weak stratum is "same employer and start, kept apart", but it's only 0.26 percent of ids. A second judge pass flagged three pairs with start after end, which led me to two real bugs: a null title acting as a wildcard and a null end erasing known ends. My own hand labelling found the garbled text and missing fields the judges had passed.

没标准答案 → 分层抽样 + 两个评委 + 自己手标；弱点那一类只占 0.26% 的 id；第二轮判官标出 3 对「开始晚于结束」，顺着查出两个真 bug（空头衔当永久通配、空结束日期抹掉已知结束日期 4.3 万次）；乱码和缺字段是我自己手标时发现的，判官放过去了。

不用评委的检查（直接在输出上数数）

不需要标准答案、全量都能算，每改一次规则马上知道变好还是变坏。

检查	怎么数	终版结果	说明什么
覆盖率	输入里每个「人 + 抓取日期」，输出里有没有一份履历	99.94%	缺的 0.06% 是整次快照每行都坏、被丢掉的（3,900 万次里 2,539 次）
主键唯一	(user_id, asof_ts, position_id) 有没有重复	0 重复	同一天同一个职位不会出现两行
倒序日期	输出里有没有职位开始晚于结束	0 个	合并时取值不会拼出不可能的日期（修之前出现过 733 个）
可疑拆分	同一个人、同一个抓取日期、同一个雇主、同一个开始日期，却有 ≥2 个不同 id 的组数	3.7 万组	这种很可能是一份工作被拆成两个 id（漏合并）；剩下的大多是头衔不同的真并行职位，不一定是错

26.7 万 → 3.7 万是什么：「可疑拆分」这个数在每一版上都量一次，看它怎么一步步降下来的。是**第一版和终版的对比**，中间每一步都是一条具体的改动：

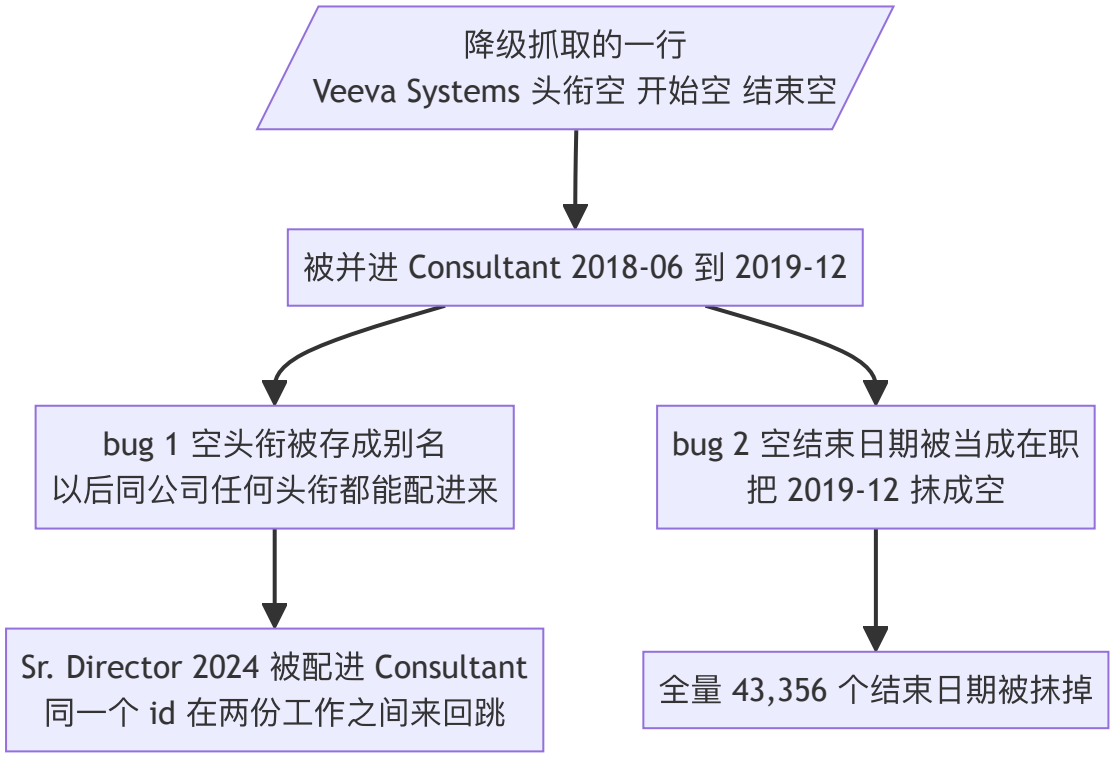
版本 / 改动	可疑拆分组数
第一版：只做精确匹配（公司名、头衔完全一样才认）	26.7 万
第一轮判官的 4 类漏配各修一条：公司改名兜底（Polar Truck）、URL 规范化（https/www 不一样）、快照内去重也用前缀判雇主（Mural Project）、去重不看结束日期（Lloyds）	15.5 万
空头衔当本行通配（降级抓取整份头衔全空）	12.6 万
后续调整、两个 bug 修复、Unicode 规范化、乱码修复	约 10.5 万
同雇主 + 同开始 + 同结束就合并（你手标 15 对全是同一份之后加的）	5.5 万
规则微调（描述相同等）	4.9 万
规则 6：折叠已发出的重复 id	**3.7 万**

These checks need no labels: 99.94% coverage, a unique primary key, no inverted dates, and suspicious splits, meaning one person, date, employer and start with two or more ids, down from 267k with exact matching to 37k, each step tied to one rule change.

判官帮我找到的两个真 bug

怎么发现的：第二轮 Claude 判官评「合并了、头衔大改」那一层，标了 3 对「不是同一份」，理由是**开始晚于结束**（Mercado Libre、Alcon、Veeva）。可输出里明明没有倒序日期 → 说明这只是症状。顺着 Veeva 那个人（user 708311970）去查输入：

- 他有一段 Consultant（2018–06 ~ 2019–12），后来是 Sr. Director（2024–10 ~ 至今）。
- 2025–09–21、11–24、12–16 有三次**降级抓取**：整份只剩一行「Veeva Systems」，头衔、开始、结束全是空的。
- 这一行被并进了 Consultant，于是同时触发了两个 bug。



	bug 1 · 空头衔变成永久通配	bug 2 · 空结束日期抹掉已知结束日期

原因	absorb 把空头衔 "" 也加进了职位的头衔别名；匹配时 "" 在别名里就算头衔对上	为了防止开始晚于结束加了一个日期保护，那次改写把「空结束」当成了「在职」，覆盖了旧值；违反了「每个字段取最新非空值」
怎么修	空头衔**只在这一行里**当通配，**不存进别名**；match 里删掉 `"" in pos.title_keys` 那个判断	结束日期候选只取非空：先用这次的，没有就用上次的，都不合法才置空；同时保证开始不晚于结束
代码	state.py absorb：`if tk := title_key(row["title_raw"]): self.title_keys.add(tk)`	state.py absorb：`for end in (row["enddate"], old_end): if end is not None and (start is None or end >= start): ...`
效果	同一个 id 不再在两份工作之间跳	抹掉的结束日期 43,356 → 53（剩下的 53 是再入职，后来由规则 3 解决）
防回归	用 Veeva 这个人做了两个回归测试，以后谁改坏了测试就会挂	

判官 vs 人工各抓到什么：判官抓到这两个 bug 的症状；乱码（ä¸€æ±½–å¤§ä¼— = 一汽–大众）和评估数据缺字段是你自己手标时看到的，判官都放过去了。你的手标还推翻了一条规则（同开始同结束就合并）。

A judge flagged three pairs with start after end. Tracing one user showed a degraded scrape with no title, start or end. Its empty title was stored as an alias, so it matched any later title, and its empty end was treated as "ongoing" and erased a known end, 43,356 times in total. The fix: an empty title is a wildcard only for that row, and an empty end never overwrites a known one. Erasures dropped to 53, with two regression tests from that user.

人工评估解决了什么

解决了，而且解决的是判官解决不了的：

- **发现了乱码**：你标注时看到「ä¸€æ±½–å¤§ä¼—」，判官看了也放过去了。后来加了修复，修了 16,314 个格子。
- **发现了评估数据缺字段**，判官同样没注意到。

- **改变了一条规则**：你标的 15 对「同公司、同开始、同结束、头衔不同」全是同一份工作，于是推翻了你之前「不加」的决定。加上之后疑似漏合并从 10.5 万降到 5.5 万。
- **校准判官**：你和 Claude 判官 60 对里 52 对一致，kappa 0.66，说明判官大体可信。不一致的全在「同公司同开始却分开」那一层，你每次都判成同一份。这也证实了这一层确实是弱点。

还没解决的：「同公司同开始却分开」这一层判官之间都不一致（Claude 认为 38% 对，GPT 8%），要更多人工标注；它只占 0.26% 的 id。

Hand labelling caught what the judges missed: garbled text, fixed in 16,314 cells, and missing fields in the eval data. It also changed a rule, since all 15 same-start-and-end pairs I labelled were one job, which halved suspected missed merges from 105k to 55k. And it calibrated the judges: 52 of 60 agree with Claude, kappa 0.66, with every disagreement in the kept-apart stratum.

判官到底干什么用（没有用来训练模型）

一句话：判官只是**离线的考官**，用来给规则打分、找错，不在流水线里，也没用来训练或微调任何模型。

- **打分**：每一类决定（合并了头衔大改 / 中改、同雇主开新 id、同雇主同开始却分开）抽 75 个，问「一份还是两份」，算出每类的准确率。
- **找错**：判官说错的那些样例，我去读原数据找原因 → 改一条规则 → 加一个回归测试。两个真 bug 就是这样来的。
- **A/B 比较**：每改一条规则，把新合并、新拆开的样例交给判官，看改了之后变好还是变坏，再决定留不留。
- **两个判官 + 人工**：Claude 和 GPT 用同一个提示词，互相对照（kappa 0.66）；再拿你手标的 60 对校准判官。

唯一一次拿标注「训练」：学习打分器（`eval/scorer.py`，README 第 6 节和架构图里都写了，要讲）

- **想回答的问题**：我手定的打分（头衔相似度 + 同雇主 25 + 同开始 10 + 同结束 5）权重拍得对不对？让数据自己学一遍看看。

- **数据**：六轮判官标注去掉「拿不准」、去重，加上你手标的 60 对，共 **823 对**（587 对同一份、236 对两份）。
- **特征 13 个**：都是匹配时代码能看到的信号，这样学出来的模型能直接替换打分函数。头衔相似度、头衔完全一样、缩写、同雇主、公司名相似度、开始相同、开始差几个月、开始缺失、结束相同、结束缺失、描述相同、描述重叠、描述缺失。
- **模型**：逻辑回归，纯 Python 实现、不加依赖，5 折交叉验证（数据分 5 份，轮流拿 1 份测、4 份训，避免在训练数据上自己考自己）。
- **踩过的坑**：第一版放了「系统当时合没合」这个特征，等于把答案递给了模型（标签泄漏），分数虚高，去掉了。

5 折交叉验证	准确率	合并精确率	合并召回
手定规则（系统当时的决定）	80.0%	98.6%	72.9%
逻辑回归，阈值 0.5	90.9%	92.1%	95.4%
逻辑回归，阈值 0.8	83.1%	96.7%	79.0%

- **学出来的权重**：开始日期相同 +2.34、开始差距 −1.43、结束相同 +0.89、描述重叠 +0.56、头衔相似度只有 +0.33。也就是 **开始日期 » 结束日期 » 描述 » 头衔**，和手写规则、和你标注时的直觉（「时间开始结束差不多的基本就是一个岗位」）一致。
- **怎么读这张表**：精确率 = 合并的里面有多少真该合；召回 = 该合的里面合上了多少。阈值 0.5 时召回高很多，但精确率掉到 92%，会多出很多错合；把阈值调到 0.8，精确率和手定规则差不多（≈97%），召回只多 6 个点（72.9% → 79.0%）。
- **为什么没上线**：① 同精度下只多 6 个点，收益不大；② 样本有偏：全是系统已经做过决定的同雇主样例，负样本少，模型没见过「不同雇主」的情况；③ 要上线还得重新做 A/B 和判官评估。
- **它的价值**：独立证明手写规则的权重方向是对的（日期最重要、头衔最不可靠）。下一步标注多了，可以只在规则拿不准的那一层用它。

「为什么不直接让 LLM 判每一对」放在最后的 To sum up；「蒸馏成小打分模型」放在未来展望。

The judges are offline graders, not part of the pipeline and not used for training. They measure precision per stratum, surface failure cases that

become rule changes and regression tests, and A/B every change. The only training was a logistic-regression scorer on 823 labelled pairs, which confirmed the hand rules' ordering but wasn't shipped because the sample was biased. Calling an LLM per pair wouldn't scale to 100 billion rows and wouldn't give reproducible ids.

测试（51 个）：评估那段带一句

- **半天 + 增量 = 全量**（`test_full_run_equals_half_plus_incremental`）：把样例数据按日期从中间切成前后两半。先只跑前一半、存下状态；再读这个状态跑后一半。把两次的输出拼起来，和一次性跑全部数据比，**必须完全一样**。它同时证明了两件事：增量没算错；前一半跑的时候根本看不到后一半，结果却一样，说明全量跑时也没用到未来信息。
- **迟到重放 = 全量**（`test_late_snapshot_is_replayed_and_matches_a_full_run`）：第一天给 1 月和明年 1 月的快照，第二天才给中间 6 月那次（迟到）。系统把这个人从头重放，输出必须和一次性全量跑一样，并且把他列进 restated。真实数据上也做过：切成两天、128 个迟到的人，结果和全量一致。
- **金丝雀**（`test_canary_future_snapshot_never_leaks_backwards`）：故意塞一个 2030 年的假快照（公司叫 CANARY_CORP_XYZ）。它只能出现在 2030 那一天的输出里，而且 2030 年之前的所有输出要和没塞时逐字节一样。你当时点名说这个好。
- 其他：Case A / B 精确对上；主键唯一；每个有效的（人，日期）都有输出；id 稳定；没有倒序日期；同样输入两次输出一样；约 20 个从真实人物来的回归测试（Veeva、Febase、Honeywell、Lloyds……）。

51 tests. The key ones: a full run equals a half run plus an incremental run on the other half, which also proves no future data is used; a late snapshot replayed equals a full run; and a planted 2030 canary snapshot only appears on its own date and changes nothing before it.

试过的方案和取舍（README 第 6 节）

试了什么	结果	留不留
只存最新值来认职位	Febase 那个人：公司名从「Febase」改成「Federacao Baiana De Saude…」，几个月后又改回「Febase」。只记最新值时，改回来的名字和记着的对不上 → 开了第二个 id。**改成别名集合**	改了
只认 ASCII 字符	正则 `[^a-z0-9]` 把中文、俄文头衔整个清空成 ""，空 = 通配，22.8 万行能匹配任何头衔；Développeur 被切成 d veloppeur。改成 Unicode 规范化	修了
多语言 embedding	多并 383 组，抽 75 组 72 组对；但慢 3 倍、镜像大 1.2 GB	默认关
词表 + 中日文按字切	翻译类收益差不多，不加依赖；第一版把英文词（chief）也改写了，修了	留
同雇主 + 同开始 + 同结束就合并	多并 2,963 组，55/71 对；错的是真同时做两份。不合的错是合的 3 倍	留
描述当信号	描述相同 → 99% 是同一份；描述不同 → 仍有 82% 是同一份。只当正面证据，多并 29 组	留
学习打分器（逻辑回归）	学出来的顺序和手写规则一样，同精度召回多 6 个点，但样本有偏（见图 6）	没上线
折叠重复 id（规则 6）	965 次合并，重复组 4.9 万 → 3.7 万，52/64 对	留
再入职开新 id（规则 3）	多了 1,579 个 id；939 个「日期乱跳」的 id 降到 0；抽查 57/58 对；还快了 5%	留
迟到快照	一开始直接隔离，后来改成重放	重放

「改成别名集合」是什么意思：每个职位不只记一个公司名、一个头衔，而是把**出现过的所有**公司名、URL、头衔都存进一个集合（company_keys、

url_keys、title_keys）。以后新来一行，只要和集合里**任何一个**对上就能认出来。输出时仍然显示最新的那个值。一句话：**认人用全部历史，展示用最新值**。Febase 改回原名时，原名还在集合里，所以能认回同一个 id。

「日期乱跳」是什么：一个 id 的开始日期一会儿是 2019 一会儿是 2022，因为两段不同时期的同名工作被并成了一个 id，absorb 每次用最新的值覆盖。再入职单独开 id 之后，这种情况变成 0。

主要取舍：merge when in doubt。漏合并 = 一个重复的 id 永远带下去；错合并 = 丢一个头衔。前者更常见、更伤。

Storing only the latest value broke when someone renamed their employer and later renamed it back, so each position keeps alias sets for names, URLs and titles: match on any alias, display the latest. ASCII-only normalisation turned 228k non-Latin titles into empty strings that matched anything, so I switched to Unicode. Treating a rehire as a new position added 1,579 ids, removed all 939 ids whose dates jumped around, and was right 57 of 58 times.

To sum up · 总结

- **核心**：每个人一个状态机，按日期一次喂一个快照，所以从结构上就不可能用到未来信息。
- **匹配**：六条规则 + 打分（头衔相似度 + 同雇主 25 + 同开始 10 + 同结束 5，⩾ 55 才配），取舍是 merge when in doubt。
- **规模**：人和人独立 → 按 user_id 分桶并行，线性扩展；单机 1150 万行 75–78 秒，1000 亿行约 800 核时；Spark 跑同一个函数。
- **每天更新**：存状态接着跑，只动有新数据的人；迟到数据只重放那一个人。
- **评估**：没有标准答案 → 分层抽样 + 两个 LLM 判官 + 60 对人工 + 不用评委的检查；每个发现都变成一条规则和一个回归测试。

为什么不直接让 LLM 判每一对（面试官很可能问）

- **规模**：1000 亿行，每对都调一次 LLM，成本和时间都不现实；规则版只要约 800 核时。
- **可复现**：id 必须稳定，同样的输入每次都要给同样的结果；LLM 的回答会变。
- **可解释**：规则每一步都说得清为什么，出错能定位到是哪条规则。
- **LLM 放在哪**：放在离线评估里当考官，抽样打分、找错，不放在每一行都要走的路径上。

Why not have an LLM judge every pair? At 100 billion rows it's far too slow and expensive, and ids must be reproducible, which an LLM's answers aren't. Rules are explainable and each error points to one rule. So the LLM sits offline as a grader on samples, not on the hot path.

未来展望 · Next steps

- **蒸馏小打分模型**：等判官和人工的标注攒多了，蒸馏成一个小的打分模型，只用在「同雇主同开始却分开」这类规则拿不准的少数样例上。
- **补弱点那一层的标注**：「同雇主同开始却分开」判官之间不一致（Claude 38%、GPT 8%），需要更多人工标注和判官。
- **描述的向量**：离线预先算好描述的 embedding，匹配时只查不算，避开现在 embedding 慢 3 倍的问题。
- **Spark 上真集群**：把分区调到每个几十万行，接上增量状态和隔离；状态放按 user_id 分区、只追加的表（Iceberg / Delta）。
- **history 只追加、按需读**：每天新数据单独一个文件，只有迟到的人才按 user_id 去读，内存不再随天数变大。
- **删除名单**：被遗忘权（GDPR）请求的经历在输出时过滤掉，因为 Case A 会把用户删掉的经历一直带着。
- **小修**：韩文职位词的编码范围补上；公司名也加首字母匹配（Revelio Labs 对 RL）。

Next: distil the judge and human labels into a small scorer used only where the rules are unsure, label more of the kept–apart stratum, precompute description embeddings, run Spark on a real cluster with incremental state, make history append–only, add a deletion list for right–to–be–forgotten requests, and fix the Korean range and initials matching for employers.

怎么用 AI（README 第 7 节，被问到再讲）

- **AI 做的**：按我的设计写代码、起草测试、跑探索脚本、当判官标样例、顺着我指出的失败去追原因。
- **我做的**：每一条规则和每一个阈值、「什么算同一份工作」的定义、抽样怎么分层、60 对人工标注。
- **我主导改过的决定**：只丢坏行不丢整次快照；同开始同结束就合并（先反对，标完 60 对后改主意）；再入职开新 id；迟到数据重放；要做金丝雀测试；GDPR 删除名单进下一步。
- **最关键的发现**：乱码和评估数据缺字段是我自己读样例发现的，判官放过去了；我的标注还推翻了一条规则。

AI wrote code from my designs, drafted tests, ran exploration scripts, labelled pairs as a judge and traced failures I pointed at. I decided every rule and threshold, defined what "same job" means, designed the sampling and labelled 60 pairs by hand. The garbled text came from my own labelling, and my labels changed one of the rules.

你的项目 ↔ 系统设计书里的知识点（今晚按这张表翻书）

左边是你代码里已经做了的事，右边是书里对应的章节。面试时说「这个就是书里讲的 X」，显得你懂原理。书：Alex Xu《System Design Interview》第一册；DDIA = 《Designing Data-Intensive Applications》。

你项目里的做法	书里的知识点	去哪看	优先级
position_id = user_id × 2^16 + 序号，不用全局计数器	分布式唯一 ID 生成（Twitter Snowflake：时间戳 + 机器号 + 序号）	Alex Xu 第 7 章 Unique ID Generator	⭐⭐⭐
800 核时、12 小时 64 核、输出 1.08 倍	粗略估算（QPS、存储、算力）	Alex Xu 第 2 章 Back-of-the-envelope	⭐⭐⭐
user_id % 64 分桶，人和人独立	分区 / 分片；取模分片扩容时要大搬家 → 一致性哈希	Alex Xu 第 5 章 Consistent Hashing；DDIA 第 6 章 Partitioning	⭐⭐⭐
每天只处理有新数据的人；迟到数据只重放这个人	批处理 vs 流处理；事件时间 vs 处理时间；迟到数据；增量计算	DDIA 第 10 章 Batch、第 11 章 Stream（Reasoning About Time）	⭐⭐⭐
全量 = 半量 + 增量（测试保证）；重跑结果一样	幂等、确定性、可重放	DDIA 第 11 章 Idempotence；第 10 章 MapReduce 的确定性重算	⭐⭐
Spark mapInArrow 跑同一函数	MapReduce / 数据流引擎、按 key 分组	DDIA 第 10 章	⭐⭐
原子写文件；一个人出错回滚隔离	容错、原子性、死信队列（quarantine）	DDIA 第 7 章 Transactions（原子性）；Alex Xu 第 10 章 Notification（重试和死信）	⭐⭐
状态存 parquet；生产用按用户分区的追加写表	键值存储、数据存储结构、追加写日志	Alex Xu 第 6 章 Key-Value Store；DDIA 第 3 章 Storage（日志结构）	⭐⭐
指纹跳过重复快照（57.5% 重复）	去重、哈希、内容寻址	Alex Xu 第 9 章 Web Crawler（内容去重：哈希比较）	⭐⭐

名字规范化 + 别名集合 + 模糊匹配	实体解析 / 记录链接（书里没有，Revelio 的核心业务）	手册的系统设计章会讲 blocking、候选生成	⭐⭐⭐
读 cgroup CPU 配额决定进程数	容器资源限制、单机扩展	Alex Xu 第 1 章（纵向 vs 横向扩展）	⭐

今晚看书的顺序（约 2 小时）

1. **Alex Xu 第 7 章 唯一 ID 生成**（20 分钟）：对比你的 id 设计和 Snowflake。
   要会说：我的 id 不用中心服务，同样输入得到同样 id；缺点是每人最多 2^16 个职位，代码里有检查。
2. **Alex Xu 第 2 章 估算**（15 分钟）：照着把你的 800 核时算一遍：100B ÷ 35k 行/核秒 ≈ 2.86M 核秒 ≈ 794 核时。
3. **Alex Xu 第 5 章 一致性哈希**（20 分钟）：被问「64 桶以后要变 128 桶怎么办」时用。
   取模分桶一改桶数几乎所有人都要搬；一致性哈希只搬一小部分。你的场景是批处理，每次重新分桶成本可接受，这点要说出来。
4. **DDIA 第 11 章「时间」和「幂等」两节**（30 分钟）：事件时间、迟到数据、水位线（watermark），对应你的迟到重放。
5. **DDIA 第 10 章 MapReduce 和数据流引擎**（20 分钟）：对应 Spark 扩展。
6. 剩余时间：Alex Xu 第 6 章键值存储的「分区、复制」两节。

代码文件总表（VS Code 里按这个顺序翻）

文件	行数	干什么	关键函数
schema.py	70	列名/类型统一	conform
partition.py	33	分 64 桶	bucket_expr, partition_input
clean.py	39	七条行规则丢坏行（README 写 six，是把「开始早于 1950」「结束早于 1950」算成一条）	clean
repair.py	53	修乱码、丢失的反斜杠	repair_text
normalize.py	126	公司/头衔/URL 规范化，头衔相似度，描述重合	company_key, title_key, title_similarity
glossary.py	261	多语言职位词表，中日韩按字	translate_cjk
match.py	249	规则 1–6	dedupe, is_candidate, _score, match, consolidate
state.py	104	Position / UserState，absorb，id 生成	absorb, absorb_position, new_position
step.py	43	一个快照的状态转移	fingerprint, step
bucket.py	144	一个桶：逐人逐快照、增量、迟到重放、隔离错误	coalesce_bucket, _replay
runner.py	175	多进程跑所有桶、原子写文件、读 cgroup CPU	run, process_bucket
semantic.py	95	可选的多语言 embedding（默认关）	SemanticTitles
cli.py	45	命令行入口	main
spark/coalesce_spark.py	74	Spark 适配	mapInArrow 调 coalesce_bucket

代码原文 + 逐段讲解（只看这里就够，不用开本地）

每段先说「这段在干什么」，下面是你仓库里的原文（英文注释也是你代码里本来就有的）。行号对应 GitHub 仓库里的文件。

〇 · 读进来：schema.py / partition.py / runner.py / cli.py（流水线最前面这几步）

schema.py 里先定好的「规矩」 src/coalesce/schema.py 开头

- `URL_IN / URL_OUT`：题目文字叫 company_linkedin_url，样例数据叫 company_website，两个名字都认，输出统一叫前者。
- `TEXT_FIELDS`（5 个文本列：公司、URL、头衔、地点、描述）和 `DATE_FIELDS`（开始、结束）合起来是 `FIELDS`：这 7 个字段就是每段职位要合并、要输出的内容。
- `INPUT_SCHEMA`：输入必须有的 10 列和各自类型（抓取日期 Date、user_id Int64、5 个文本 String、开始结束 Date、档案内顺序 Int32）。
- `OUTPUT_SCHEMA`：输出 10 列，抓取日期改叫 asof_ts，去掉档案内顺序，加上 position_id。
- `MAX_TEXT_LEN = 4000`：文本最长保留 4000 字符。

```
URL_IN = "company_website"
URL_OUT = "company_linkedin_url"

# Free-text fields that travel through the pipeline unchanged except for
# "latest non-null wins" merging.
TEXT_FIELDS = ["company_raw", URL_OUT, "title_raw", "location_raw", "descript
DATE_FIELDS = ["startdate", "enddate"]
FIELDS = TEXT_FIELDS + DATE_FIELDS

INPUT_SCHEMA: dict[str, pl.DataType] = {
    "observed_ts": pl.Date,
    "user_id": pl.Int64,
    "company_raw": pl.String,
    URL_OUT: pl.String,
    "title_raw": pl.String,
    "location_raw": pl.String,
    "description": pl.String,
    "startdate": pl.Date,
    "enddate": pl.Date,
```

```
    "order_in_profile": pl.Int32,
}

OUTPUT_COLUMNS = ["asof_ts", "user_id", *FIELDS, "position_id"]
OUTPUT_SCHEMA: dict[str, pl.DataType] = {
    "asof_ts": pl.Date,
    "user_id": pl.Int64,
    **{f: pl.String for f in TEXT_FIELDS},
    **{f: pl.Date for f in DATE_FIELDS},
    "position_id": pl.Int64,
}

# Guard against pathological rows blowing up memory; longer text is truncated
MAX_TEXT_LEN = 4000


class SchemaError(ValueError):
    """Raised when the input does not look like the assignment's input table.
```

conform src/coalesce/schema.py 第 50–70 行

把任何一份输入整理成标准样子，四步：① 只有 company_website 时改名成 company_linkedin_url；② 对照 10 列，缺哪列就直接报错（SchemaError），不带着错数据往下跑；③ 逐列转类型：日期先按「微秒时间戳」读再截成日期，所以给日期、时间戳都能接；`strict=False` 表示转不了的值变成空值，交给后面清洗去丢；④ 文本只留前 4000 字符，防止个别巨长的行撑爆内存。整个过程是 LazyFrame（惰性执行），真正读文件时才执行，而且只读需要的列。Spark 版本调用的也是它，所以两边输出一模一样。

```
def conform(lf: pl.LazyFrame) -> pl.LazyFrame:
    """Rename, type-coerce and truncate an input frame to ``INPUT_SCHEMA``."""
    cols = set(lf.collect_schema().names())
    if URL_IN in cols and URL_OUT not in cols:
        lf = lf.rename({URL_IN: URL_OUT})
        cols = (cols - {URL_IN}) | {URL_OUT}
    missing = set(INPUT_SCHEMA) - cols
    if missing:
        raise SchemaError(f"input is missing columns: {sorted(missing)}")
    casts = []
    for name, dtype in INPUT_SCHEMA.items():
        expr = pl.col(name)
        if dtype == pl.Date:
```

```
            # Accept datetimes/timestamps too; the assignment only needs day
            expr = expr.cast(pl.Datetime("us"), strict=False).cast(pl.Date)
        else:
            expr = expr.cast(dtype, strict=False)
        if dtype == pl.String:
            expr = expr.str.slice(0, MAX_TEXT_LEN)
        casts.append(expr.alias(name))
    return lf.select(casts)
```

`bucket_expr` src/coalesce/partition.py 第 15–18 行

算每一行属于哪个桶：`abs(user_id) % n_buckets`。同一个人永远进同一个桶。用自己写的取模而不是 Polars 自带的 hash，因为后者换版本会变，而状态文件是按桶存的，第二天同一个人必须回到同一个桶。

```
def bucket_expr(n_buckets: int) -> pl.Expr:
    """``user_id % n``: a hash we own, so a user lands in the same bucket (an
    same state file) across Polars versions. ``Expr.hash`` is not stable acro
    return (pl.col("user_id").abs() % n_buckets).cast(pl.Int32).alias("bucket
```

`partition_input` src/coalesce/partition.py 第 21–33 行

把输入切成 64 个桶：先建好 64 个文件夹 `bucket=0 … bucket=63`；然后**一次只读一个输入文件**，按桶号拆开，每一份写成一个小 parquet，写完立刻释放内存（`del df`）。所以不管输入多大，这一步的内存上限都只是「一个输入文件」。

```
def partition_input(input_dir: Path, work_dir: Path, n_buckets: int) -> list[
    """Write ``work_dir/bucket=<k>/part-<i>.parquet``; return the bucket dire
    files = sorted(input_dir.glob("*.parquet"))
    if not files:
        raise FileNotFoundError(f"no parquet files in {input_dir}")
    for k in range(n_buckets):
        (work_dir / f"bucket={k}").mkdir(parents=True, exist_ok=True)
    for i, f in enumerate(files):
        df = conform(pl.scan_parquet(f)).with_columns(bucket_expr(n_buckets))
        for (k,), part in df.partition_by("bucket", as_dict=True).items():
            part.drop("bucket").write_parquet(work_dir / f"bucket={k}" / f"pa
        del df  # keep the parent's footprint to one input file at a time
    return [work_dir / f"bucket={k}" for k in range(n_buckets)]
```

process_bucket src/coalesce/runner.py 第 69–100 行

每个工作进程处理一个桶：读这个桶的所有小文件（空桶也照样处理，因为它的状态要往下带）；有 `--state-in` 就读这个桶上次存下的状态；调 `coalesce_bucket`；然后用「先写临时文件再改名」的原子写，写出这个桶的输出、隔离名单、重放名单，以及新的状态和 history。

```
def process_bucket(job: BucketJob) -> dict:
    """Worker: coalesce one bucket and write its outputs. Returns metrics."""
    k, bucket_dir, out_dir, state_in, state_out, report_dir = job
    t0 = time.perf_counter()
    files = sorted(bucket_dir.glob("*.parquet"))
    # A bucket can be empty (small input, quiet day); it must still carry its
    df = pl.read_parquet(files) if files else pl.DataFrame(schema=INPUT_SCHEM
    states = history = None
    if state_in is not None:
        states = load_state(
            _read_or_empty(state_in / f"positions-{k:04d}.parquet", STATE_POS
            _read_or_empty(state_in / f"users-{k:04d}.parquet", STATE_USER_SC
        )
    if state_out is not None:
        # Keep the cleaned input so late data can be replayed. Rewritten per
        # simplicity; in production this is an append-only, user-partitioned
        history = (_read_or_empty(state_in / f"history-{k:04d}.parquet", INPU
                   if state_in is not None else pl.DataFrame(schema=INPUT_SCH
    res = coalesce_bucket(df, states, history)
    _write_atomic(res.output, out_dir / f"part-{k:04d}.parquet")
    if res.quarantine.height and report_dir is not None:
        _write_atomic(res.quarantine, report_dir / "quarantine" / f"part-{k:0
    if res.restated and report_dir is not None:
        _write_atomic(pl.DataFrame({"user_id": res.restated}, schema={"user_i
                      report_dir / "restated" / f"part-{k:04d}.parquet")
    if state_out is not None:
        _write_atomic(res.positions, state_out / f"positions-{k:04d}.parquet"
        _write_atomic(res.users, state_out / f"users-{k:04d}.parquet")
        _write_atomic(res.history, state_out / f"history-{k:04d}.parquet")
    res.metrics.update(bucket=k, input_rows=df.height, quarantined_users=res.
                       seconds=round(time.perf_counter() - t0, 2))
    return res.metrics
```

run src/coalesce/runner.py 第 112–175 行

总调度：① 检查上次状态的桶数和这次一致（不一致就拒绝）；② 调 `partition_input` 切桶；③ 每个桶包成一个任务；④ 设 `POLARS_MAX_THREADS=1`（每个进程只用 1 个 Polars 线程，因为逐人循环是 Python 在跑，N 个进程各开一整套线程池会抢 CPU，实测慢 2.7 倍）；⑤ 进程数 = `available_cpus()`，用进程池并行跑所有桶；⑥ 删掉临时文件夹；⑦ 写 `_meta.json` 记桶数；⑧ 把每个桶的指标加总（输入行数、丢掉的行按规则分、修了几格、输出行数、隔离几人、耗时）写进 run.json。

```
def run(input_dir: Path, output_dir: Path, *, n_buckets: int = 64, workers: i
        state_in: Path | None = None, state_out: Path | None = None,
        work_dir: Path | None = None, report_dir: Path | None = None) -> dict
    """Partition the input by user, coalesce the buckets in parallel, write o

    ``output_dir`` receives only the output parquet files, so any reader can
    one dataset. Metrics and quarantined users go to ``report_dir`` when give
    Returns the run metrics.
    """
    t0 = time.perf_counter()
    output_dir.mkdir(parents=True, exist_ok=True)
    if work_dir is not None:
        work_dir.mkdir(parents=True, exist_ok=True)
    if state_in is not None:
        _check_state_layout(state_in, n_buckets)
    tmp_root = Path(tempfile.mkdtemp(prefix="coalesce-", dir=work_dir))
    saved_threads = os.environ.get("POLARS_MAX_THREADS")
    try:
        buckets = partition_input(input_dir, tmp_root, n_buckets)
        t_part = time.perf_counter() - t0
        jobs = [BucketJob(k, b, output_dir, state_in, state_out, report_dir)
        # One Polars thread per worker: the per-user loop is Python-bound, an
        # each spawning a full thread pool oversubscribes the CPUs (measured
        # Spawned children read the env at import, so set it before creating
        os.environ["POLARS_MAX_THREADS"] = "1"
        workers = workers or available_cpus()
        with ProcessPoolExecutor(max_workers=workers,
                                 mp_context=multiprocessing.get_context("spaw
            per_bucket = list(ex.map(process_bucket, jobs))
    finally:
        shutil.rmtree(tmp_root, ignore_errors=True)
        if saved_threads is None:
            os.environ.pop("POLARS_MAX_THREADS", None)
        else:
            os.environ["POLARS_MAX_THREADS"] = saved_threads
    if state_out is not None:
        (state_out / "_meta.json").write_text(json.dumps({"n_buckets": n_buck

    dropped: dict[str, int] = {}
    for m in per_bucket:
```

```
        for rule, n in m["dropped_rows"].items():
            dropped[rule] = dropped.get(rule, 0) + n
    metrics = {
        "input_rows": sum(m["input_rows"] for m in per_bucket),
        "kept_rows": sum(m["kept_rows"] for m in per_bucket),
        "repaired_text_cells": sum(m["repaired_text_cells"] for m in per_buck
        "output_rows": sum(m["output_rows"] for m in per_bucket),
        "users": sum(m["users"] for m in per_bucket),
        "snapshots": sum(m["snapshots"] for m in per_bucket),
        "late_snapshots": sum(m["late_snapshots"] for m in per_bucket),
        "restated_users": sum(m["restated_users"] for m in per_bucket),
        "positions_merged": sum(m.get("positions_merged", 0) for m in per_buc
        "quarantined_users": sum(m["quarantined_users"] for m in per_bucket),
        "dropped_rows": dropped,
        "n_buckets": n_buckets,
        "workers": workers,
        "seconds_partition": round(t_part, 2),
        "seconds_total": round(time.perf_counter() - t0, 2),
        "buckets": per_bucket,
    }
    if report_dir is not None:
        report_dir.mkdir(parents=True, exist_ok=True)
        (report_dir / "run.json").write_text(json.dumps(metrics, indent=2))
    return metrics
```

main src/coalesce/cli.py 第 17–41 行

命令行入口：读参数 `--input --output --buckets(默认 64) --workers --state-in --state-out --work-dir --report-dir`，调 `run()`，把汇总指标打到 stderr；有迟到重放的人或被隔离的人就额外打印提示。

```
def main(argv: list[str] | None = None) -> int:
    p = argparse.ArgumentParser(description="Coalesce job-profile observation
    p.add_argument("--input", type=Path, default=Path("/input"))
    p.add_argument("--output", type=Path, default=Path("/output"))
    p.add_argument("--buckets", type=int, default=64, help="hash partitions o
    p.add_argument("--workers", type=int, default=None, help="parallel proces
    p.add_argument("--state-in", type=Path, default=None, help="continue from
    p.add_argument("--state-out", type=Path, default=None, help="save per-use
    p.add_argument("--work-dir", type=Path, default=None, help="scratch dir f
    p.add_argument("--report-dir", type=Path, default=None,
                   help="write run.json metrics and quarantined users here (d
    a = p.parse_args(argv)
    m = run(a.input, a.output, n_buckets=a.buckets, workers=a.workers,
            state_in=a.state_in, state_out=a.state_out, work_dir=a.work_dir,
    summary = {k: v for k, v in m.items() if k != "buckets"}
```

```
    print(json.dumps(summary, indent=2), file=sys.stderr)
    if m["restated_users"]:
        where = (f"listed in {a.report_dir}/restated/" if a.report_dir
                 else "rerun with --report-dir to list them")
        print(f"note: {m['restated_users']} user(s) received late data and we
              f"their earlier rows with this run's ({where})", file=sys.stder
    if m["quarantined_users"]:
        where = f"see {a.report_dir}/quarantine/" if a.report_dir else "rerun
        print(f"warning: {m['quarantined_users']} user(s) quarantined; {where
    return 0
```

一 · 核心：step.py

`fingerprint` src/coalesce/step.py 第 16–20 行

给一次快照算一个指纹：所有行排好序拼起来再哈希。和上一次指纹一样，就说明这次抓取跟上次一模一样（57.5% 的快照是这样），直接跳过匹配，只换个日期再输出。用 blake2b 而不用 Python 自带的 hash()，因为后者每次进程启动都会变，存进状态里第二天就对不上了。

```
def fingerprint(rows: list[dict]) -> int:
    """Stable across processes (unlike ``hash``), so it can live in saved sta
    # repr per row so None/str/date mix sorts without TypeError; order-insens
    canon = "\n".join(sorted(repr(tuple(r[f] for f in FIELDS)) for r in rows)
    return int.from_bytes(hashlib.blake2b(canon.encode(), digest_size=8).dige
```

`step` src/coalesce/step.py 第 23–43 行

整个项目的心脏。输入：这个人的状态、这次的日期、这次抓到的所有行。顺序：指纹判断 → 规则 1 去重 → 规则 2–5 匹配 → 没配上的开新 id → absorb 更新字段 → 规则 6 折叠重复 → 输出这个人当前所有职位（没抓到的也输出，这就是 Case A）。因为它只看「这一次 + 之前的状态」，所以结构上不可能用到未来信息。

```
def step(state: UserState, observed_ts: dt.date, rows: list[dict]) -> list[tu
    """Advance ``state`` with one snapshot; return the output rows for ``asof

    Output row layout matches ``schema.OUTPUT_COLUMNS``.
```

```
    """
    fp = fingerprint(rows)
    if fp != state.last_fingerprint:
        rows = dedupe(rows)
        assigned = match(rows, state.positions)
        touched: set[int] = set()
        for i, row in enumerate(rows):
            pos = assigned[i] if i in assigned else state.new_position()
            pos.absorb(row)
            touched.add(pos.position_id)
        # Duplicates that were issued earlier (company spelled two ways, a re
        # arrived before we could tell) are folded as soon as a snapshot touc
        state.merges += consolidate(state.positions, touched)
        state.last_fingerprint = fp
    state.last_ts = observed_ts
    uid = state.user_id
    return [(observed_ts, uid, *p.values(), p.position_id) for p in state.pos
```

二 · 匹配规则：match.py

dedupe src/coalesce/match.py 第 106–127 行

规则 1：同一次抓取里的重复行合并。条件：起始日期相同（或一边没有）、同雇主，再加头衔一样/相似 ⩾90/描述一样/结束日期一样之一。合并时「第一个非空值」赢。每组记住所有合进来的头衔，这样第三行就算只和第二行头衔一样也能找到这一组。

```
def dedupe(rows: list[dict]) -> list[dict]:
    """Merge rows within one snapshot that are the same job; first non-null v

    Each merged group remembers every title key folded into it, so a third ro
    equals the *second* row's title still finds the group.
    """
    groups: list[tuple[dict, set[str]]] = []
    for row in rows:
        tk = title_key(row["title_raw"])
        for group, keys in groups:
            if not (_same_start(row["startdate"], group["startdate"]) and sam
                continue
            if _same_title(tk, keys, row, group):
                for f in FIELDS:
                    if group[f] is None:
                        group[f] = row[f]
                if tk:
                    keys.add(tk)
```

```
                break
        else:
            groups.append((dict(row), {tk} if tk else set()))
    return [group for group, _ in groups]
```

starts_after_end src/coalesce/match.py 第 134–137 行

规则 3：这行的开始日期晚于这段已知的结束日期 → 是重新入职（或季节性工作），不能当成同一段的修改。

```
def starts_after_end(row: dict, pos: Position) -> bool:
    """The row's stint begins after the position is known to have ended: a la
    at the same employer (a rehire, a seasonal job), never an edit of the ear
    return row["startdate"] is not None and pos.enddate is not None and row["
```

is_candidate src/coalesce/match.py 第 140–151 行

规则 2：判断一行能不能作为某段已知职位的候选。同雇主时：头衔它用过、或起始日期兼容（±2 个月或只有年份）就算候选。不同雇主时只有一种例外：头衔完全一样且起始日期完全一样，这是公司改名（例子写在注释里：Polar Truck → Polar Transportes – Grupo DHL）。空头衔当通配符。

```
def is_candidate(row: dict, pos: Position) -> bool:
    """Rule 2: same employer and (compatible start or a title the position ha
    or, with no shared employer, the identical title starting the same month
    A row that starts after the position ended is never a candidate (a rehire
    if starts_after_end(row, pos):
        return False
    tk = title_key(row["title_raw"])
    title_same = tk in pos.title_keys or not tk  # a missing title is a wildc
    if same_company(row, pos):
        return title_same or start_compatible(row["startdate"], pos.startdate
    # "Polar Truck" -> "Polar Transportes – Grupo DHL" with no URL on either
    return title_same and row["startdate"] is not None and row["startdate"] =
```

_score src/coalesce/match.py 第 154–168 行

规则 5 的打分：头衔相似度（0–100，取和这段所有历史头衔里最高的）+ 同雇主 25（头衔完全一样再 +2 打破平局）+ 同起始 10 + 同结束 5。相似度不够高且开了可选 embedding 时，再用 embedding 补一次。

```
def _score(row: dict, pos: Position) -> int:
    """Rule 4 score: title similarity (0-100) plus bonuses for employer, star
    tk = title_key(row["title_raw"])
    s = max(title_similarity(tk, t) for t in pos.title_keys) if pos.title_key
    if s < DEDUPE_TITLE_SIM and semantic.enabled():  # optional embedding mod
        s = max(semantic.combined_title_similarity(tk, t, s) for t in pos.tit
    if same_company(row, pos):
        # The rename fallback scores 100 + 10 = 110, so a same-employer candi
        # down to title similarity 76; exact alias breaks ties with a transla
        s += SAME_EMPLOYER_BONUS + (2 if tk and tk in pos.title_keys else 0)
    if row["startdate"] is not None and row["startdate"] == pos.startdate:
        s += 10
    if row["enddate"] is not None and row["enddate"] == pos.enddate:
        s += 5
    return s
```

match src/coalesce/match.py 第 171–202 行

规则 4 和 5：先给每一行列出所有候选，同时记录每个职位被哪些行想要。第一轮：只有一个候选、而且这个候选也只被这一行想要 → 直接配。第二轮：剩下的所有（行，职位）对按分数从高到低贪心配对；低于 55 停止；职位已被占时只有分数 ⩾120（同雇主 + 头衔一样，明显是抓取重复）才还能配进去。

```
def match(rows: list[dict], positions: list[Position]) -> dict[int, Position]
    """Return ``{row_index: position}`` for rows that match an existing posit
    cands: dict[int, list[Position]] = {}
    wanted_by: dict[int, list[int]] = {}
    for i, row in enumerate(rows):
        for pos in positions:
            if is_candidate(row, pos):
                cands.setdefault(i, []).append(pos)
                wanted_by.setdefault(pos.position_id, []).append(i)

    assigned: dict[int, Position] = {}
    taken: set[int] = set()
    # Rule 3: unique, uncontested candidate.
    for i, ps in cands.items():
        if len(ps) == 1 and wanted_by[ps[0].position_id] == [i]:
            assigned[i] = ps[0]
            taken.add(ps[0].position_id)
```

```
    # Rules 4 & 5: greedy by score for the contested rest; ties keep row orde
    scored = sorted(
        ((_score(rows[i], p), i, p) for i, ps in cands.items() if i not in as
        key=lambda t: (-t[0], t[1]),
    )
    for s, i, p in scored:
        if s < AMBIGUOUS_MIN_SCORE:
            break
        if i in assigned:
            continue
        if p.position_id in taken and s < DUPLICATE_SCORE:
            continue
        assigned[i] = p
        taken.add(p.position_id)
    return assigned
```

same_position src/coalesce/match.py 第 209–222 行

规则 6 判断两个已经发出去的职位是不是同一段：起始日期必须相同、雇主（任意别名）重叠，再加结束日期一样 / 头衔相似 ≥90 / 描述一样之一。

```
def same_position(p: Position, q: Position) -> bool:
    """Rule 1 applied to two positions: same employer (any alias), same expli
    and one of the same end (both ongoing counts), a title alias pair at leas
    ``DEDUPE_TITLE_SIM`` similar, or the same description."""
    if p.startdate is None or p.startdate != q.startdate:
        return False
    if not employers_overlap(p.url_keys, p.company_keys, q.url_keys, q.compan
        return False
    if p.enddate == q.enddate:
        return True
    if p.title_keys and q.title_keys and max(
            title_similarity(a, b) for a in p.title_keys for b in q.title_key
        return True
    return same_description(p.description, q.description)
```

consolidate src/coalesce/match.py 第 236–249 行

规则 6：把重复的职位折叠进更老的 id（id 小的更老），新的那个 id 永远退休、不会再发。只检查这次快照碰到过的职位，所以很快。字段值用「最近被更新的那个」。

```
def consolidate(positions: list[Position], touched: set[int]) -> int:
    """Fold duplicate positions into the older id; the newer id is dropped an

    Mutates ``positions`` and ``touched`` (a survivor counts as touched). Ret
    """
    merged = 0
    while (pair := _find_duplicate(positions, touched)) is not None:
        keep, drop = pair
        drop_is_fresher = drop.position_id in touched and keep.position_id no
        keep.absorb_position(drop, other_is_fresher=drop_is_fresher)
        positions.remove(drop)
        touched.add(keep.position_id)
        merged += 1
    return merged
```

三 · 状态：state.py

absorb src/coalesce/state.py 第 43–68 行

一行配给某段职位后，用这一行更新它：每个字段「最新的非空值」赢；公司名、URL、头衔都加进别名集合。两个关键保护：① 这一行的结束日期是空的，不能抹掉已知结束（以前抹掉过 43,000 个）；② 空头衔不进别名，否则一次坏抓取会让这段职位以后吞掉这家公司的所有行。还保证永远不输出「开始晚于结束」。

```
    def absorb(self, row: dict) -> None:
        """Latest non-null value wins for every field; aliases accumulate."""
        old_end = self.enddate
        for f in FIELDS:
            v = row[f]
            if v is not None:
                setattr(self, f, v)
        # Dates may now come from two different observations. Never emit star
        # prefer this observation's end, then the previous one, else "ongoing
        # A null end in the row is "no information", not "ongoing": it must n
        # a known end (43,000 end dates were wiped this way before the guard)
        for end in (row["enddate"], old_end):
            if end is not None and (self.startdate is None or end >= self.sta
                self.enddate = end
                break
        else:
            self.enddate = None
        if (ck := company_key(row["company_raw"])) is not None:
            self.company_keys.add(ck)
```

```
        if (uk := url_key(row["company_linkedin_url"])) is not None:
            self.url_keys.add(uk)
        # A missing title is a per-row wildcard, never a permanent alias: oth
        # degraded scrape makes the position match every later row at that em
        # (seen in the data: a 2018 Consultant entry swallowed a 2024 Sr. Dir
        if tk := title_key(row["title_raw"]):
            self.title_keys.add(tk)
```

absorb_position src/coalesce/state.py 第 73–84 行

规则 6 折叠时用：别名取并集，字段值取更新的那一方；如果合完开始晚于结束，就把结束置空。

```
    def absorb_position(self, other: Position, other_is_fresher: bool) -> Non
        """Fold a duplicate position into this one. Aliases are unioned; fiel
        values come from whichever position was updated more recently."""
        for f in FIELDS:
            mine, theirs = getattr(self, f), getattr(other, f)
            if theirs is not None and (mine is None or other_is_fresher):
                setattr(self, f, theirs)
        if self.startdate is not None and self.enddate is not None and self.e
            self.enddate = None
        self.company_keys |= other.company_keys
        self.url_keys |= other.url_keys
        self.title_keys |= other.title_keys
```

new_position src/coalesce/state.py 第 96–104 行

发一个新 id：user_id 左移 16 位再或上序号，也就是 user_id × 65536 + 序号。不需要全局计数器，同样输入永远得到同样 id；每人最多 65536 段，超了就报错。

```
    def new_position(self) -> Position:
        if not 0 <= self.user_id < (1 << (63 - SEQ_BITS)):
            raise ValueError(f"user_id {self.user_id} does not fit the positi
        if self.next_seq >= MAX_SEQ:
            raise TooManyPositionsError(f"user {self.user_id} exceeds {MAX_SE
        pos = Position(position_id=(self.user_id << SEQ_BITS) | self.next_seq
        self.next_seq += 1
```

```
        self.positions.append(pos)
        return pos
```

四 · 每个桶、增量、迟到：bucket.py

`_replay` src/coalesce/bucket.py 第 78–84 行

迟到数据时用：把这个人的全部历史行 + 新行合在一起，按日期从头重放一遍。

```
def _replay(uid: int, rows: pl.DataFrame, out: list[tuple]) -> UserState:
    """Rebuild one user from scratch over all of their snapshots, in date ord
    state = UserState(uid)
    rows = rows.unique(maintain_order=True).sort(["observed_ts", "order_in_pr
    for (ts,), sdf in rows.group_by("observed_ts", maintain_order=True):
        out.extend(step(state, ts, sdf.rows(named=True)))
    return state
```

`coalesce_bucket` src/coalesce/bucket.py 第 87–144 行

处理一个桶里的所有人。先清洗，再按人、按日期排序，逐人逐快照调 step。三个要点：① 增量：有存下的状态就接着走；② 迟到：新快照比状态还旧，且有历史 → 这个人整体重放，列进 restated；没历史就隔离；③ 容错：一个人报错，回滚到他进来时的状态（checkpoint），记进 quarantine 带 traceback，不影响别人。

```
def coalesce_bucket(df: pl.DataFrame, states: dict[int, UserState] | None = N
                    history: pl.DataFrame | None = None) -> BucketResult:
    """Coalesce every user in ``df``, continuing from ``states`` when given (

    ``history`` holds the cleaned rows of earlier runs. When it is given, a u
    snapshots are not all newer than their state (late or re-delivered data)
    from scratch over history + new rows, and *all* of their as-of rows are e
    (listed in ``restated``; their position ids may change). Without it such
    quarantined as ``late_snapshot``. The returned ``history`` adds this run'
    """
    if states is None:
        states = {}
    df, clean_counts = clean(df)
    df = df.sort(["user_id", "observed_ts", "order_in_profile"])
```

```
    out: list[tuple] = []
    quarantine: list[tuple] = []
    restated: list[int] = []
    snapshots = skipped = 0
    for (uid,), udf in df.group_by("user_id", maintain_order=True):
        state = states[uid] if uid in states else UserState(uid)
        late = state.last_ts is not None and udf["observed_ts"].min() <= stat
        # If this user fails half-way, roll back to the state they arrived wi
        # earlier positions keep their ids and no half-updated rows are emitt
        checkpoint, out_mark = copy.deepcopy(state), len(out)
        try:
            if late and history is not None:
                state = _replay(uid, pl.concat([history.filter(pl.col("user_i
                restated.append(uid)
                snapshots += udf["observed_ts"].n_unique()
            else:
                for (ts,), sdf in udf.group_by("observed_ts", maintain_order=
                    if state.last_ts is not None and ts <= state.last_ts:
                        skipped += 1
                        quarantine.append((uid, "late_snapshot", f"{ts} <= st
                        continue
                    out.extend(step(state, ts, sdf.rows(named=True)))
                    snapshots += 1
        except Exception as exc:  # isolate one user's failure from the rest
            quarantine.append((uid, type(exc).__name__, traceback.format_exc(
            del out[out_mark:]
            if uid in states:
                states[uid] = checkpoint
            continue
        states[uid] = state

    positions, users = dump_state(states)
    return BucketResult(
        output=pl.DataFrame(out, schema=OUTPUT_SCHEMA, orient="row"),
        positions=positions,
        users=users,
        quarantine=pl.DataFrame(quarantine, schema=QUARANTINE_SCHEMA, orient=
        metrics={**clean_counts, "snapshots": snapshots, "late_snapshots": sk
                 "restated_users": len(restated), "users": len(states), "outp
                 "positions_merged": sum(s.merges for s in states.values())},
        history=None if history is None else pl.concat([history, df]),
        restated=restated,
    )
```

五 · 规范化和相似度：normalize.py

`company_key` src/coalesce/normalize.py 第 35–42 行

公司名规范化：Unicode 统一（全角转半角、大小写折叠）、去标点、去法律后缀（inc、llc、ltd、gmbh……）。lru_cache 缓存，同一个名字只算一次。

```
@lru_cache(maxsize=1 << 16)
def company_key(name: str | None) -> str | None:
    """Lower-case, strip punctuation and legal suffixes. ``None`` stays ``Non
    if name is None:
        return None
    low = _fold(name)
    tokens = [t for t in _NON_ALNUM.sub(" ", low).split() if t not in _COMPAN
    return " ".join(tokens) or _NON_ALNUM.sub(" ", low).strip() or None
```

`title_key` src/coalesce/normalize.py 第 45–52 行

头衔的精确键：只做大小写和标点统一，故意不翻译。注释解释了原因：如果把 Vendedora online 和 Vendedora 都翻成 sales，两段不相关的工作会看起来一样、互相抢行。

```
@lru_cache(maxsize=1 << 16)
def title_key(title: str | None) -> str:
    """Exact key: used for alias equality. No translation here on purpose: co
    'Vendedora online' and 'Vendedora' both to 'sales' would make unrelated p
    look identical and steal each other's rows."""
    if title is None:
        return ""
    return " ".join(_NON_ALNUM.sub(" ", _fold(title)).split())
```

`sim_key` src/coalesce/normalize.py 第 55–61 行

头衔的相似度键：用词表翻译成英文、去停用词、中日韩每个字一个 token。只在算相似度时用，不用来判断相等。

```
@lru_cache(maxsize=1 << 16)
def sim_key(key: str) -> str:
    """Similarity key: glossary-translated, stopwords dropped, CJK one char p
    Only ever compared with another sim_key inside ``title_similarity``."""
    text = _CJK_CHAR.sub(r" \1 ", translate_cjk(key))
```

```
    tokens = [TOKENS.get(t, t) for t in text.split()]
    return " ".join(t for t in tokens if t not in STOPWORDS) or key
```

`url_key` src/coalesce/normalize.py 第 64–72 行

URL 规范化：去 http、www、问号后面的参数和结尾斜杠，这样不同写法的同一个领英公司页算同一个。

```
@lru_cache(maxsize=1 << 16)
def url_key(url: str | None) -> str | None:
    """``https://www.linkedin.com/company/x/`` and ``linkedin.com/company/x``
    if url is None:
        return None
    u = url.strip().lower().split("?", 1)[0]
    u = re.sub(r"^https?://", "", u)
    u = re.sub(r"^www\.", "", u)
    return u.rstrip("/") or None
```

`title_similarity` src/coalesce/normalize.py 第 79–93 行

头衔相似度 0–100：完全一样或一边为空 = 100；缩写对上（mle vs machine learning engineer）= 90；翻译后的相似度键一样 = 100；否则用 rapidfuzz 的 token_set_ratio。

```
def title_similarity(a: str, b: str) -> int:
    """0..100. Exact match 100, acronym match (``mle`` vs ``machine learning

    A missing title is a wildcard (100): degraded scrapes drop every title bu
    company and dates, and such a row should still fold into the job it belon
    """
    if a == b or not a or not b:
        return 100
    ca, cb = a.replace(" ", ""), b.replace(" ", "")
    if len(ca) <= 6 and ca == _initials(b) or len(cb) <= 6 and cb == _initial
        return 90
    sa, sb = sim_key(a), sim_key(b)
    if sa == sb:
        return 100
    return int(fuzz.token_set_ratio(sa, sb))
```

六 · 清洗和修乱码：clean.py / repair.py

clean src/coalesce/clean.py 第 29–39 行

行级清洗规则：关键字段为空、公司和头衔都空、结束早于开始、日期早于 1950、开始超过抓取日一年后、结束超过两年后 → 丢这一行。每条规则单独计数写进指标。只丢行，整个快照只有在所有行都坏时才消失。

```
def clean(df: pl.DataFrame) -> tuple[pl.DataFrame, dict]:
    """Return the kept rows and ``{"dropped_rows": {rule: n}, "kept_rows": n,

    A row violating several rules is counted once per rule it violates.
    """
    flags = df.select([expr.fill_null(False).alias(name) for name, expr in RU
    dropped = {name: int(flags[name].sum()) for name in RULES}
    keep = ~pl.any_horizontal(pl.all())
    kept = df.filter(flags.select(keep).to_series())
    kept, repaired = repair_columns(kept, TEXT_FIELDS)
    return kept, {"dropped_rows": dropped, "kept_rows": kept.height, "repaire
```

repair_text src/coalesce/repair.py 第 32–39 行

修两种文字损坏：① 乱码（UTF-8 被当成 cp1252 解了一遍，ä¸€æ±½ 其实是一汽）：按 cp1252 编码回去再按 UTF-8 解，失败就说明不是乱码、原样保留；② 丢了反斜杠的转义（Lloydu2019s 其实是 Lloyd's）。都是无损、确定性的。

```
def repair_text(s: str) -> str | None:
    """Return the repaired string, or ``None`` when nothing applies."""
    out = s
    if re.search(MOJIBAKE_RE, out):
        with contextlib.suppress(UnicodeEncodeError, UnicodeDecodeError):
            out = _undo_cp1252(out).decode("utf-8")
    out = ESCAPE_RE.sub(lambda m: chr(int(m.group(1), 16)), out)
    return out if out != s else None
```

七 · 并行和容器：runner.py

available_cpus src/coalesce/runner.py 第 37–46 行

算这个进程真正能用几个 CPU：在 docker --cpus 4 里，os.cpu_count() 还是报宿主机的核数，曾经开了 14 个进程、内存是 2.5 倍。所以去读 cgroup 的 CPU 配额，取较小值。

```
def available_cpus() -> int:
    """CPUs this process may actually use.

    ``os.cpu_count()`` reports the host's cores even inside ``docker run --cp
    the real limit is the cgroup quota. Using the host count spawned 14 worke
    4-CPU quota in testing (oversubscribed, ~2.5x the memory).
    """
    n = len(os.sched_getaffinity(0)) if hasattr(os, "sched_getaffinity") else
    limit = _cgroup_cpu_limit()
    return max(1, min(n, int(limit))) if limit else n
```

八 · Spark：spark/coalesce_spark.py

`coalesce_arrow` spark/coalesce_spark.py 第 32–40 行

执行器端：一个桶的数据以 Arrow 格式进来，转成 Polars，走和单机一模一样的 conform + coalesce_bucket，再以 Arrow 输出。所以结果和单机逐字节一致。

```
def coalesce_arrow(batches: Iterator[pa.RecordBatch]) -> Iterator[pa.RecordBa
    """Executor-side: one bucket in (as Arrow batches), one bucket out."""
    table = pa.Table.from_batches(list(batches))
    if table.num_rows == 0:
        return
    # Same validation/truncation as the single-machine path, so outputs are b
    df = conform(pl.from_arrow(table).lazy()).collect()
    out = coalesce_bucket(df).output
    yield from out.to_arrow().cast(OUTPUT_ARROW).to_batches()
```

`main` spark/coalesce_spark.py 第 43–62 行

Driver 端：读 parquet，统一列名，按 user_id repartition（同一个人的行必须落在同一个分区），mapInArrow 调上面的函数，写出。注释里写了：增量状态和 quarantine 在 Spark 版里没接，单机版展示了做法。

```
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--input", required=True)
    ap.add_argument("--output", required=True)
    ap.add_argument("--buckets", type=int, default=4096)
    a = ap.parse_args()

    spark = SparkSession.builder.appName("position-coalescing").getOrCreate()
    df = spark.read.parquet(a.input)
    # Same column conventions as the single-machine path (company_website ->
    if URL_IN in df.columns and URL_OUT not in df.columns:
        df = df.withColumnRenamed(URL_IN, URL_OUT)
    df = df.select(*INPUT_SCHEMA)

    # All of a user's rows must land in one partition; which partition does n
    # one-shot run (ids come from user_id + first-seen order). Incremental st
    # quarantine output are not wired here; the single-machine runner shows t
    bucketed = df.repartition(a.buckets, F.col("user_id"))
    result = bucketed.mapInArrow(coalesce_arrow, schema=_spark_schema())
    result.write.mode("overwrite").parquet(a.output)
```

三个被问到要能当场指出代码的细节

1. **空的结束日期不能抹掉已知结束**（`state.py · absorb`）：一开始把 null 当成「还在职」，抹掉了 43,000 个结束日期。现在 null 只代表「这次没信息」。

2. **空头衔只在这一行当通配符，不进别名**（`state.py · absorb` 末尾）：不然一次坏抓取会让这段职位以后匹配这家公司的所有行（真实例子：2018 年的 Consultant 吞掉了 2024 年的 Sr. Director）。

3. **一个人出错不影响别人**（`bucket.py` 的 try/except + checkpoint）：回滚到他进来时的状态，记进 quarantine，带 traceback。